CONSOLIDATED PATENT SUBMISSION WITH ADDENDUMS

Church-Turing Meta-Machine: Complete Patent Portfolio

Inventor: Kenneth James Hamer-Hodges

Prepared for: Patent Attorney Review

Date: April 2026

COVER LETTER

Dear Attorney,

This document consolidates the complete CTMM patent portfolio into a single submission for your review. It comprises the unified base patent (which already incorporates the initial filing and the Pure Church continuation-in-part) plus three subsequent addendums, each extending the architecture with novel innovations discovered during the ongoing development process.

Document Structure

Section	Filing	Content
Part I	February 2026	Base Patent — Dual-Gate TSB, Golden Tokens, LAMBDA instru...
Addendum A	March 2026	Universal Computation Target — CLOOMC++ multi-language co
Addendum B	March 2026	Abstract GT I/O and Network Addressing — Hardware-routed ...
Addendum C	April 2026	Lambda Recursion and Self-Invocation — O(1) recursion via...

Key Architectural Progression

The addendums represent a natural evolution of the architecture:

1. Base patent establishes the security foundation: Golden Tokens, dual gates, domain purity, LAMBDA instruction, Pure Church variant.
2. Addendum A demonstrates the architecture is a universal computation target — multiple programming languages compile to the same 20-instruction capability-secured ISA. The hardware cannot distinguish the source language.
3. Addendum B extends the Golden Token type system: Abstract GTs (type 11_2) provide hardware-routed I/O, network tunneling, and structural service scoping — all through

the same GT mechanism that secures memory access.

4. Addendum C reveals LAMBDA's full power: idempotent self-invocation via CR6 achieves O(1) recursion (entry, context switch, exit) with zero additional hardware — just a one-line refinement of the existing FAULT condition. CALL CR6 recursion is shown to provide no security benefit over LAMBDA CR6 (same method, same domain, same capabilities) while imposing O(N) cost. An English natural language front-end extends the universal target to three paradigms.

Total Claims Summary

Section	Independent Claims	Dependent Claims	Total
Base Patent (Part I)	See existing filing	See existing filing	Per existing filing
Addendum A	4	3	7
Addendum B	5	4	9
Addendum C	4	3	7

Please review the addendums for filing as continuations-in-part of the base patent.

Regards,

Kenneth James Hamer-Hodges

PART I: BASE PATENT (Filed February 2026)

Church-Turing Meta-Machine: Hardware-Enforced Capability Security Through Dual Trusted Gates, Lambda Calculus Integration, and Architectural Vulnerability Elimination

---

Inventor: Kenneth James Hamer-Hodges

Filing Date: February 2026

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Lambda Calculus with the LAMBDA Instruction, NULL Capability Type, and Atomic Abstraction Architecture (Filed February 12, 2026)

Continuation-in-Part: Pure Church Lambda Processor: Architectural Exclusion of Turing-Domain Instructions as a Security Enforcement Mechanism (Filed February 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; Lambda Calculus Processor; Vulnerability Elimination by Construction; Trusted Security Base; Interactive Programming Model

---

TITLE OF THE INVENTION

Church-Turing Meta-Machine: Dual-Gate Trusted Security Base with Hardware-Enforced Lambda Calculus, Deterministic Garbage Collection, and Architectural Vulnerability Elimination in a Capability-Based Processor

CROSS-REFERENCE TO RELATED APPLICATIONS

This application consolidates and extends two related filings:

1. The CTMM patent application filed February 12, 2026, which discloses the Golden Token capability architecture, domain purity enforcement separating Turing-domain (R, W, X) from Church-domain (L, S, E) permissions, the LAMBDA instruction for lightweight in-scope code application, and the atomic abstraction architecture.
2. The continuation-in-part filed February 2026, which discloses the Pure Church Lambda Machine — demonstrating that the Turing domain can be entirely eliminated from the software instruction set without loss of computational completeness, and that this elimination constitutes a novel security enforcement mechanism.

The present submission extends both disclosures with the dual-gate Trusted Security Base (mLoad + mSave), the B (Bind) bit for capability propagation control, the F (Far/Foreign) flag for network transparency, the unified PP250 deterministic garbage collection mechanism, and the complete DATA object architecture bridging Church and Turing domains.

FIELD OF THE INVENTION

The present invention relates to a processor architecture that enforces all access control through unforgeable capability tokens (Golden Tokens) validated by a dual-gate Trusted Security Base comprising a read gate (mLoad) and a write gate (mSave). The architecture integrates Church's lambda calculus with Turing's computational model through clean domain separation, provides deterministic garbage collection through a bidirectional G-bit mechanism, and in its pure Church variant eliminates all Turing-domain instructions from the software instruction set to achieve vulnerability elimination by construction.

BACKGROUND OF THE INVENTION

The Security Problem

Contemporary processor architectures, derived from von Neumann's 1945 stored-program

design, lack hardware-enforced boundaries between code and data, between programs, and between privilege levels. Software-based security mechanisms (access control lists, virtual memory page tables, privilege rings) are bolt-on mitigations that have repeatedly proven insufficient against buffer overflows, return-oriented programming (ROP), use-after-free exploits, privilege escalation, code injection, and confused deputy attacks.

The Capability Limitation

Capability-based architectures (Cambridge CAP, IBM System/38, CHERI, and the PP250) enforce access control through unforgeable tokens, requiring valid capabilities for every memory access. However, all prior capability architectures retain Turing-domain instructions for computation. While capabilities prevent unauthorized access to memory regions, they do not prevent misuse of legitimate access. Furthermore, all prior capability architectures focus exclusively on the read path — validating capabilities when they are used — without providing symmetric validation on the write path when capabilities are propagated to other protection domains.

The Trusted Computing Base Problem

In conventional systems, the "trusted computing base" includes an operating system kernel, a hypervisor, privileged CPU modes, and memory management units — millions of lines of code that attackers can exploit. Even formally verified microkernels (seL4: ~10,000 lines) are orders of magnitude larger than necessary. The present invention reduces the entire trusted computing base to two hardware gates totalling approximately 329 lines of synthesizable HDL — five orders of magnitude smaller than Linux, two orders of magnitude smaller than seL4.

The Capability Propagation Problem

Prior capability architectures do not control capability propagation at the hardware level. Once a capability is granted, the holder can freely copy it to other protection domains, creating uncontrolled capability distribution. The present invention introduces a B (Bind) bit on namespace entries that controls whether a capability can be saved to another C-List, with hardware enforcement through the mSave write gate.

The Discovery: Pure Church Computational Completeness

The parent CTMM application discloses domain purity enforcement: Golden Token permissions are separated into Turing domain (R, W, X) and Church domain (L, S, E), and a GT cannot have permissions from both domains simultaneously. The present invention recognizes that domain purity can be extended to its logical conclusion: an entire processor can operate with only Church-domain instructions available to software, achieving computational completeness through Church-encoded lambda calculus and eliminating entire classes of vulnerabilities by construction.

Prior Art — PP250 Capability Architecture

The present invention builds upon prior work in capability-based computer architecture developed principally at Plessey Telecommunications and subsequently at ITT/Standard Electric Corporation during the period 1969–1984:

DE2000066A1 — "Data processing arrangement" (Cotton, Plessey, priority 1969-01-02). Establishes the principle of function-level execution control within segmented memory.

DE2126206C3 — "Data processing device with memory protection arrangement" (Cotton, Cole, Plessey, priority 1970-05-26). Discloses capability registers mediating all memory access — the architectural ancestor of Golden Tokens.

DE2303596C2 — "Data processing arrangement" (Cotton, Plessey, priority 1972-01-26). Extends the capability register concept with refined access control semantics.

US3771146A — "Data processing system interrupt arrangements" (Cotton, Williams, Cosserat, Plessey, priority 1972-01-26, granted 1973-11-06). Discloses secure context switching when interrupts occur during capability-protected execution.

CA945264A — "Program interrupt facilities in data processing systems" (Cotton, Plessey, priority 1970-09-02, granted 1974-04-09). Establishes mechanisms for asynchronous event handling within capability-protected execution environments.

US3657736A — "Method of assembling subroutines" (Boom, Plessey, priority 1969-01-02, granted 1972-04-18). Establishes the principle of structured data passing between computational units.

DE2230830C2 — "Data processing system" (Arnold, Boom, Plessey, priority 1971-06-24, granted 1985-03-21). Establishes the architectural framework for multi-module capability-based systems.

US4121286A — "Memory space allocation and deallocation arrangements" (Venton, Plessey, priority 1975-10-08, granted 1978-10-17). Discloses the mechanism for deallocating master capability table entries when storage blocks are returned — directly relevant to the present invention's deterministic garbage collection mechanism.

MY8400351A — "Information flow security mechanisms for data processing systems" (Hamer-Hodges, Plessey, priority 1976-07-30). Establishes the principle of hardware-enforced information flow control.

CA1132251A — "Data handling equipment for use with sequential access digital data storage" (Hamer-Hodges, priority 1976-11-17, granted 1982-09-21). Addresses peripheral integration within capability-protected systems.

HK31983A — "Information protection arrangements in data processing systems" (Hamer-Hodges, Plessey, priority 1977-05-04). Extends the security model with additional protection mechanisms.

DE2909762A1 — "Remote communication system" (Cotton, ITT/Standard Electric, priority 1978-03-17). Extends capability-based principles to telecommunications switching.

DD143994A5 — "Telecommunications switching system" (Lawrence, ITT/Standard Electric, priority 1979-04-05). Applies capability-based design principles to switching network architecture.

Distinction from Prior Art

The above prior art establishes capability registers, capability-mediated memory access, multi-processor capability systems, capability deallocation, and information flow security. However, none of the prior art discloses or suggests:

1. A dual-gate Trusted Security Base (mLoad read gate + mSave write gate) providing symmetric validation on both read and write paths for capability operations
2. A B (Bind) bit on namespace entries controlling capability propagation through the mSave write gate
3. A NULL capability type within the GT type field for safe initialization, revocation, and garbage collection
4. A lightweight LAMBDA instruction for in-scope code application with machine-status fast path
5. Self-describing stack frames with a 1-bit tag distinguishing LAMBDA from CALL frames
6. Architectural exclusion of Turing-domain instructions as a security enforcement mechanism
7. Deterministic garbage collection with bidirectional G-bit integrated into both mLoad and mSave validation paths
8. A unified address space where memory, attached devices, and machine registers are segments of one flat space, all protected by the same GT gate via mLoad

Prior Art Distinction — Academic Capability Architectures

System	Dual TSB Gates	Lambda Reduction	Capability Security	Turing Excluded	B-bit Propagation Control
Cambridge CAP	No	No	Yes	No	No
IBM System/38	No	No	Yes	No	No
Intel iAPX 432	No	No	Yes	No	No
CHERI (Cambridge)	No	No	Yes	No	No
LISP Machines	No	Yes	No	No	No
Reduceron	No	Yes	No	No	No
CTMM (this invention)	Yes	Yes	Yes	Yes (Pure Church variant)	Yes

---

SUMMARY OF THE INVENTION

The present invention provides a processor architecture, the Church-Turing Meta-Machine (CTMM), that implements capability-based security through a dual-gate

Trusted Security Base and integrates Church's lambda calculus with Turing's computational model:

1. Golden Token (GT) Architecture

A 2-bit Type field in every capability token classifies four categories: Inform (local reference, Type = 00), Outform (remote reference, Type = 01), NULL (empty/invalid, Type = 10), and Abstract (unforgeable constant value, e.g., pi, Type = 11). Six permission bits enforce domain purity: Turing domain (R, W, X) and Church domain (L, S, E), never mixed. Capability registers hold capabilities exclusively; data registers hold values exclusively.

2. Dual-Gate Trusted Security Base

mLoad — The Read Gate: Every read-side instruction routes through mLoad for GT validation (version, seal, bounds) and permission checking. Permission gate table: R→DREAD, W→DWRITE, X→LAMBDA, L→LOAD, S→SAVE (c-list access), E→CALL. M-elevation bypasses permission checks.

mSave — The Write Gate: Every write to a c-list routes through mSave for source GT validation: version match, seal valid, B=1 (bindable), and F-bit detection on the target slot (F=1 means FAR/foreign object requiring HTTP/tunnel access). Symmetric counterpart to mLoad in the TSB.

3. B (Bind) Bit — Capability Propagation Control

Namespace entry word1 bit 31. mSave requires B=1 on the source GT before committing to a c-list. Defaults to 0 — "no bind by default." CALL auto-clears B on all preserved CRs passed to the callee. Allow Bind is the explicit special case via TPERM before CALL (e.g., TPERM CR0, EB). This prevents uncontrolled capability distribution across protection domains.

4. NULL Type

A capability register encoding that represents an empty, invalid, or revoked capability. Any operation on a NULL-typed GT causes an immediate FAULT. NULL enables clean initialization, safe revocation, and unambiguous garbage collection scanning.

5. LAMBDA Instruction

A dedicated hardware instruction for Church's function application: LAMBDA CRn, x. CRn holds a Golden Token with X (Execute) permission pointing to a code body in the same protection domain. LAMBDA saves only the return address to a machine status register and branches to the code body. Unlike CALL (E permission, domain crossing, full stack frame), LAMBDA stays within the current protection domain with near-zero overhead.

6. Machine-Status Fast Path

In the common case (LAMBDA → body → RETURN), the return address and LAMBDA-active flag live in machine status registers, not the capability stack. RETURN checks the LAMBDA-active flag: if set, it restores PC from the machine status register with zero stack access. When a CALL or CHANGE intervenes during a LAMBDA body, the LAMBDA state (LAMBDA_PC and LAMBDA-active flag) is saved to the CALL stack frame and the LAMBDA-active flag is cleared. When RETURN pops that CALL frame, it restores the saved LAMBDA state, so the next RETURN correctly uses the LAMBDA fast path.

7. Self-Describing Stack Frames

Every stack frame carries a 1-bit tag: CALL frame (0) or LAMBDA frame (1). RETURN inspects the tag to determine whether to perform full domain restoration (CALL) or simple PC restoration (LAMBDA).

8. Deterministic Garbage Collection (PP250)

A four-phase Scan-Identify-Clear-Flip process with bidirectional G-bit. GC is a safe Turing abstraction — an atomic Turing machine hidden behind a Church-callable namespace entry, entered via CALL, exited via RETURN. The G-bit is integrated into both mLoad and mSave validation paths, ensuring that every namespace access contributes to liveness tracking regardless of the instruction or permission type.

9. Pure Church Lambda Processor Variant

An entire processor operating with only six Church-domain instructions (LOAD, SAVE, CALL, RETURN, LAMBDA, TPERM), architecturally excluding all Turing-domain instructions. Computational completeness is achieved through Church-encoded lambda calculus: Church numerals, Church booleans, Church pairs, and Y-combinator recursion. This eliminates buffer overflows, ROP attacks, code injection, and privilege escalation by construction.

10. Atomic Abstraction Architecture

No central OS, VM, privileged mode, or superuser. All system services are atomic abstractions accessed via Golden Tokens, with mLoad as the single trusted gate. Safe Turing abstractions hide Turing implementations inside Church-callable entries — Church is the armor (interface, security), Turing is the sword inside (implementation, hidden and atomic).

11. Unified Address Space

Memory (MSB 0x00-0xFD), attached devices (MSB 0xFE), and machine register bank (MSB 0xFF) are all segments of one flat address space, all protected by the same GT gate via mLoad. Without the right GT, any address range is unreachable.

DETAILED DESCRIPTION OF THE INVENTION

1. Architecture Overview

The CTMM is a processor architecture built on the principle that every computational resource — code, data, I/O, network objects, cryptographic keys — is accessed exclusively through unforgeable capability tokens called Golden Tokens (GTs). The architecture integrates two foundational computational models with clean separation:

- Turing's model: Data registers hold numeric values and perform arithmetic, logic, comparison, and branching. Data registers hold values — and only values.
- Church's model: Capability registers hold Golden Tokens that name, protect, and mediate access to every resource. Capability registers hold capabilities — and only capabilities.

The synthesis is expressed as the Church-Lambda-Object-Oriented-Meta-Calculus (CLOOMC), which organizes GT permissions into two mutually exclusive domains:

Domain	Permissions	Purpose
Turing	R (Read), W (Write), X (Execute)	Data access and code execution
Church	L (Load), S (Save), E (Enter)	Capability transfer and domain crossing

A seventh permission, M (Meta/Microcode), is transient — elevated on CRs by microcode to perform privileged actions, then cleared when the microcode operation completes. M is never stored in the GT itself. No user instruction can set, test, or observe M.

2. The Golden Token Format

2.1 Sim-32 Format (32-bit Implementation)

```
GT [31:0]:
  [31:25] Version     (7 bits)  — Namespace entry version for cross-check
  [24:8]  Index       (17 bits) — Namespace table index
  [7:2]   Permissions (6 bits)  — R, W, X, L, S, E
  [1:0]   Type        (2 bits)  — Resource classification
```

Each capability register is 128 bits wide (4 x 32-bit words):

Word	Content
word0	The 32-bit Golden Token
word1	Location (from namespace entry) + B-bit (bit 31)
word2	Limit (from namespace entry)
word3	VersionSeals: Version(7) + FNV Seal(25)

2.2 Sim-64 Format (64-bit Implementation)

```
GT [63:0]:
  [31:0]  Offset      (32 bits) — Namespace entry offset
  [56:32] Spare       (25 bits) — Reserved / version counter
```

```
  [57]    G           (1 bit)   — Garbage collection mark bit
  [63:58] Permissions (6 bits)  — R, W, X, L, S, E
```

2.3 GT Type Field

Value	Type	Semantic Category	Hardware Behavior
00	Inform	Name (local)	Dereference through mLoad: validate MAC, version, p
01	Outform	Name (remote)	Dereference through HTTPS fetch/flush or RPC tunnel
10	NULL	Empty/invalid	FAULT on any operation
11	Abstract	Unforgeable constant (e.g., pi)	Returns encoded value; immutable; no namespace d

3. The Dual-Gate Trusted Security Base

The CTMM architecture's security is enforced by two hardware gates that form the complete Trusted Security Base (TSB). Every capability operation must pass through one or both gates. The total TSB is approximately 329 lines of synthesizable Amaranth HDL.

3.1 mLoad — The Read Gate

mLoad is the sole trusted path for all capability register writes that involve namespace dereferencing. Every Church instruction that reads from the namespace routes through mLoad.

mLoad Validation Pipeline:

```
mLoad(source_capability, required_permission, index, destCR):

  1. Permission Check
     Does the source capability have the required permission (L, E, or M)?
     Failure → FAULT

  2. Bounds Check
     Is the index within the C-List/namespace range?
     Failure → FAULT

  3. Fetch Golden Token
     Read the GT from the C-List at the given index.

  4. Version Match
     Does the GT's version match the namespace entry's version?
     Failure → FAULT (stale token — entry was GC'd and recycled)

  5. MAC/Seal Validation
     Recompute FNV seal from Location + Limit.
     Does it match the stored seal?
     Failure → FAULT (tampered namespace entry)

  6. G-bit Reset
     Clear G=0 on the accessed namespace entry.
     Unconditional — reachability determines liveness.

  7. Write to Destination CR
     Write the full capability to the destination register.
     This is the SOLE path for all CR writes.

  8. Thread Table Shadow Update
     Write the CR to Thread[CRd] in the thread table.
```

```
     Keeps the thread table continuously current.
```

Permission Gate Table (mLoad):

Permission	Instruction	Operation
R	DREAD	Read data from namespace entry
W	DWRITE	Write data to namespace entry
X	LAMBDA	Execute code in same domain
L	LOAD	Load capability from C-List
S	SAVE	Access C-List for save operation
E	CALL	Enter abstraction / cross domain

3.2 mSave — The Write Gate

mSave is the symmetric counterpart to mLoad, validating every write of a Golden Token to a C-List. Where mLoad gates what you can read, mSave gates what you can propagate.

mSave Validation Pipeline:

```
mSave(source_GT, target_clist, target_index):

  1. Source GT Version Match
     Does the source GT's version match its namespace entry's version?
     Failure → FAULT (stale capability cannot be propagated)

  2. Source GT Seal Validation
     Recompute FNV seal from source's Location + Limit.
     Does it match the stored seal?
     Failure → FAULT (tampered source)

  3. Target C-List Bounds Check
     Is the destination slot index within the C-List's allocated range?
     Failure → FAULT (out-of-bounds write prevented)

  4. B-bit Check
     Is B=1 on the source GT's namespace entry (word1, bit 31)?
     Failure → FAULT (capability is not bindable — cannot be saved)

  5. F-bit Detection on Target Slot
     Is F=1 on the target namespace entry?
     If yes → route through HTTP/tunnel for remote object access
     If no → local write proceeds

  6. Seal Recomputation
     Compute new FNV seal from the written Location + Limit values.
     Construct new VersionSeals: existing version + new seal.

  7. G-bit Reset
     Clear G=0 on the accessed namespace entries.
     Ensures GC liveness tracking on write path.

  8. Commit Write
     Write the capability to the target C-List slot.
```

3.3 The Dual-Gate Guarantee

In both implementations, the security guarantee is identical:

No instruction, no microcode sequence, no hardware path can read a Golden Token from the namespace without passing through mLoad's complete validation pipeline. No instruction can write a Golden Token to a C-List without passing through mSave's complete validation pipeline. If either gate rejects an operation, it faults. Period.

This dual-gate architecture is novel: all prior capability systems validate only on the read path. The CTMM validates on both read and write paths, with the B-bit providing hardware-enforced control over capability propagation.

4. The B (Bind) Bit

The B bit is a namespace entry metadata flag (word1, bit 31) that controls whether a capability can be propagated — saved to another C-List. B is not a GT permission bit; it is a property of the namespace entry.

Key Properties:

- Default B=0: Capabilities are non-bindable by default. This is the secure default — a capability can be used but not shared unless explicitly permitted.
- CALL Auto-Clears B: When a CALL instruction preserves CRs for the callee, B is automatically cleared on all preserved capabilities. The callee can use the capabilities but cannot propagate them to other domains.
- Allow Bind via TPERM: To make a capability bindable, the holder must explicitly use TPERM with the B modifier (e.g., TPERM CR0, EB) before CALL. This makes bind-permission an explicit, auditable action.
- mSave Enforces B: The mSave write gate checks B=1 on the source GT before committing the write. If B=0, the save FAULTs.

This mechanism solves the capability propagation problem: a service can be given a capability to use (B=0) without granting the ability to share it with others. Only explicit bind-permission (B=1) enables propagation.

5. The LAMBDA Instruction

5.1 Instruction Format

```
LAMBDA CRn, x
```

- CRn: A capability register holding a Golden Token with X (Execute) permission pointing to executable code in the same protection domain.
- x: A data register holding the argument value.

5.2 Execution Sequence

```
Step 1: Verify CRn.Type = Inform (00) → FAULT if NULL, Outform, or Abstract
Step 2: Check X permission on CRn → FAULT if X bit not set
Step 3: Check LAMBDA-active flag → FAULT if already set (non-nestable)
Step 4: Save return address (PC+4) to LAMBDA_PC machine status register
```

```
Step 5: Set LAMBDA-active flag
Step 6: Branch to CRn's code entry point
Step 7: Body executes using data registers for computation
Step 8: Body executes RETURN → hardware detects LAMBDA-active flag
Step 9: Restore PC from LAMBDA_PC, clear LAMBDA-active flag
```

5.3 Key Distinction from CALL

Property	LAMBDA	CALL
Permission required	X (Execute)	E (Enter)
Protection domain	Same (no C-List change)	Crosses to new domain
Stack frame (common case)	None (machine status)	Full (CRs + PC + LAMBDA state)
mLoad validation	None (body already validated)	Full path for new C-List
B-bit behavior	N/A (no capability passing)	Auto-clears B on preserved CRs
CR writes	None	mLoad writes CRs during C-List switch
Overhead	~2 cycles	10+ cycles

5.4 Non-Nestable LAMBDA with CALL-Mediated Nesting

A second LAMBDA while LAMBDA-active is set causes a FAULT. However, a CALL during a LAMBDA body saves the LAMBDA machine status as part of its stack frame, clears the LAMBDA-active flag, and permits a nested LAMBDA within the called procedure. This provides controlled nesting through the existing CALL/RETURN infrastructure.

6. Namespace Entry Structure

Each namespace entry describes a resource with three words:

```
Namespace Entry:
  Word 1: Location (base address) + B-bit (bit 31) + F-bit
  Word 2: Limit (bounds)
  Word 3: VersionSeals = Version(7) + Seal(25)
```

Metadata Flags (NOT in the GT):

Flag	Location	Description
B (Bind)	Word1, bit 31	Whether this capability can be saved to another C-List. D...
F (Far/Foreign)	Word1 metadata	Whether this entry references a remote resource. Triggers...
G (Garbage)	Implicit in version	GC liveness flag, managed by mLoad/mSave pipelines.

7. DATA Objects — Bridging Church and Turing Domains

DATA objects are namespace entries accessed via DREAD/DWRITE Turing instructions with R/W permission checks and bounds validation. They bridge the Church and Turing domains:

- Church domain provides the capability (GT with R or W permission) that names and authorizes access to the DATA object.
- Turing domain provides the instructions (DREAD, DWRITE) that read and write the data within the authorized bounds.

- mLoad validates the GT's R or W permission before any data access proceeds.
- Bounds checking ensures all reads and writes fall within the namespace entry's Location..Limit range.

8. Safe Turing Abstractions

The architecture supports hidden Turing implementations inside Church-callable entries. Church is the armor (interface, security), Turing is the sword inside (implementation, hidden and atomic).

- Entered only via CALL/LAMBDA with valid GTs
- Exited only via RETURN
- Internal Turing instructions (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR) are invisible to the caller
- The caller sees only a Church interface — CALL(Abstraction.Method(args))
- GC is itself a safe Turing abstraction

Minimal Turing ISA (inside safe abstractions): DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR + shared RETURN — 11 integer-only instructions, no FP (FP is Church-domain via abstractions).

9. Deterministic Garbage Collection (PP250)

9.1 Four-Phase Cycle

1. Scan: Walk the reachability tree from all live roots (CRs, call stack, thread table), clearing G=0 on reachable entries via mLoad.
2. Identify: Entries still marked G=1 after scan are unreachable.
3. Clear: Reclaim unreachable entries.
4. Flip: Bump version on reclaimed entries, instantly invalidating all outstanding GTs that reference the old version.

9.2 Bidirectional G-bit Integration

The G-bit is reset on every namespace access through both mLoad (read path) and mSave (write path). This ensures that reachability determines liveness regardless of whether the access is a read or write operation.

9.3 GC as Safe Turing Abstraction

Garbage collection is a safe Turing abstraction — an atomic Turing machine hidden behind a Church-callable namespace entry. It is entered via CALL, executes Turing-domain scanning internally, and exits via RETURN. The caller sees only a Church interface.

10. Network Transparency

10.1 Outform GTs and the F-bit

Outform GTs (Type = 01) reference remote resources. The F (Far/Foreign) flag on namespace entries marks foreign objects requiring HTTP/tunnel access:

- R on Outform: Object fetch via standard HTTPS GET
- W on Outform: Object flush via standard HTTPS PUT
- E on Outform: RPC call through encrypted point-to-point tunnel

10.2 Tunnel Key Storage

RPC tunnel keys are stored in standard namespace entries accessed via CAP.LOAD with R permission. Both communicating machines hold matching namespace entries with identical key material in the Location and Limit fields. Garbage collection of the namespace entry (version bump) instantly revokes the tunnel.

11. Three Dispatch Styles

Abstractions can resolve method calls via:

(a) Symbolic Resolver (high-security): CR14 contains a dispatcher that reads symbolic method names from CR6 and resolves them at runtime. Maximum isolation — the caller never sees code addresses.

(b) LAMBDA Fast-Path (performance): CR14 uses the LAMBDA instruction to jump directly to method bodies with X permission. Near-zero overhead (~2-3 cycles per invocation).

(c) Traditional Compiled Binary (fastest): CR14 contains a conventional code object with method offsets.

All three styles present the same interface to the caller. The caller cannot determine which style is used.

12. Unified Address Space

Memory (MSB 0x00-0xFD), attached devices (MSB 0xFE), and machine register bank (MSB 0xFF) are all segments of one flat address space. Every segment is protected by the same GT gate via mLoad. Without the right Golden Token, any address range — whether RAM, a UART register, or a machine status register — is unreachable. No separate I/O instructions or privilege modes are needed.

13. Atomic Abstraction Architecture

The CTMM eliminates four architectural pillars that every major cyberattack exploits:

1. No central operating system: All system services are atomic abstractions accessed through Golden Tokens.
2. No virtual memory: Namespace entries are the memory model.

3. No privileged hardware mode: mLoad is the single trusted gate — nobody bypasses it.
4. No superuser / root: No identity has universal access.

These four eliminations produce the architecture's 7 Zeroes:

Zero	Property
1	Zero OS required
2	Zero virtual memory
3	Zero privilege escalation
4	Zero superuser / root
5	Zero unauthorized code execution
6	Zero unauthorized data access
7	Zero containment escape

14. Instruction Encoding

32-bit fixed-width: opcode[5] | cond[4] | dst[4] | src[4] | imm[15]. The 5-bit opcode supports 20 instructions (10 Church + 10 Turing). ARM-style conditional execution via 4-bit condition codes (N, Z, C, V).

15. Boot Sequence

Five-phase hardware boot:

Phase	Action
0 IDLE	Waiting for boot_start
1 FAULT_RST	Clear all CRs to NULL, all DRs to zero
2 LOAD_NS	Load namespace GT into CR15 (one hardwired GT)
3 INIT_THRD	Initialize CR8 (thread), CR5 (services)
4 LOAD_NUC	Load CR14 (nucleus), CR6 (C-List)
5 COMPLETE	Begin instruction fetch

Phase 1 sets all capability registers to NULL, providing clean initialization. Phases 2-4 load valid GTs through mLoad (except CR15, the one hardwired bootstrap GT).

REDUCTION TO PRACTICE

Hardware Proof: Synthesizable FPGA Implementation

The architecture is implemented in two hardware description languages:

Amaranth HDL (~3,150 lines, 18 modules): Synthesized to Verilog (29,000 lines) and successfully placed on an iCE40 HX8K FPGA target: 1,982 LUTs (26% utilization), 1,132 flip-flops, 10 BRAMs (31%).

SystemVerilog: Parallel hardware implementation providing the same architectural coverage.

Sim-32 (RV32-Cap): A 32-bit GT system based on RISC-V RV32I with custom Church extensions, implemented as a web simulator demonstrating all architectural features including the dual-gate TSB, B-bit enforcement, and PP250 garbage collection.

Pure Church Machine: A standalone Church-only 32-bit processor with 10 opcodes (including fused ELOADCALL and XLOADLAMBDA for cycle reduction), implementing ARM-style conditional execution.

Software Proof: HP-35 Scientific Calculator

179 Church-domain instructions implementing the complete HP-35 (digit entry, four-function arithmetic, trigonometry via Taylor series, logarithms, exponentiation, square root via Newton-Y-combinator iteration, stack management, constant retrieval) — zero Turing-domain instructions. Verified by automated parsing.

Software Proof: SlideRule Arithmetic Engine

98 Church-domain instructions implementing 9 arithmetic operations (ADD, SUB, MUL, DIV, MOD, LOG, EXP, SQRT, POW) — zero Turing-domain instructions. Notable: SQRT uses Y-combinator-driven Newton iteration; MOD is SUB(a, MUL(b, DIV(a, b))) — composing Church primitives without any Turing instruction.

Software Proof: Interactive Church Computer REPL

A Haskell implementation (~1,000 lines, 6 modules) providing an interactive programming environment for the Pure Church Machine. Supports Ada Lovelace-style variable bindings, conventional mathematical notation, and demonstrates computational completeness through a Bernoulli sum-of-squares program (17 named intermediate results, verifying $1^2+2^2+3^2+4^2=30$ two ways). Any Turing-domain instruction produces an immediate FAULT.

Web Simulators

Three web-based simulators (CTMM Sim-64, RV32-Cap Sim-32, Pure Church Machine) demonstrate all architectural features interactively, including the dual-gate TSB, B-bit enforcement, PP250 garbage collection, LAMBDA execution, and cross-architecture tunnel messaging.

CLAIMS

Claim 1 — GT Type Field with NULL and Abstract Types

A processor architecture comprising a capability register file wherein each register holds a Golden Token (GT) having a Type field of at least two bits that architecturally classifies the token content as one of: a local reference (Inform, Type = 00), a remote reference (Outform, Type = 01), a null/empty/invalid capability (NULL, Type = 10), or an unforgeable constant value (Abstract, Type = 11, e.g., mathematical or physical constants such as pi); wherein the Type field is checked by hardware at each instruction to determine the execution path; wherein a NULL-typed token causes an immediate hardware fault on any operation, providing an unambiguous representation for empty, uninitialized, or revoked capability registers; and wherein an Abstract-typed token encodes an immutable value that requires no namespace dereference, enabling hardware-protected constants that cannot be forged, modified, or confused with capabilities.

Claim 2 — NULL Type for Initialization, Revocation, and Garbage Collection

The architecture of Claim 1, wherein the NULL type serves three architectural roles:

(a) initialization, wherein freshly created threads have all non-essential capability registers set to NULL, providing a clean and unambiguous initial state;

(b) revocation, wherein revoking a capability sets the corresponding register to NULL, causing any subsequent use to FAULT immediately rather than encountering ambiguous state;

(c) garbage collection, wherein the GC scanner distinguishes NULL-typed registers (no namespace entry to scan) from valid Inform or Outform GTs (namespace entries to mark as reachable), eliminating the ambiguity between an empty register and a valid capability with index zero.

Claim 3 — LAMBDA Instruction

The architecture of Claim 1, further comprising a LAMBDA instruction having two operands: a capability register (CRn) holding a Golden Token with Execute (X) permission referencing executable code, and a data register (x) holding an argument value; wherein said LAMBDA instruction:

(a) verifies that CRn holds a token of Inform type (00) with Execute (X) permission;

(b) saves the return address (PC+4) to a machine status register;

(c) sets a LAMBDA-active flag in machine status;

(d) branches to the code referenced by CRn for execution within the same protection domain, without changing the current capability list, without allocating a stack frame, and without performing namespace revalidation;

(e) allows the code body to operate on argument and result values through data registers without writing to capability registers;

thereby implementing Church's lambda calculus function application as a lightweight in-scope code invocation at reduced overhead compared to a domain-crossing CALL instruction, achieving macro-like code reuse without code duplication.

Claim 4 — LAMBDA vs. CALL Distinction

The architecture of Claim 3, wherein the LAMBDA instruction uses Execute (X) permission and operates within the current protection domain without stack frame allocation or capability list switching; and wherein a separate CALL instruction uses Enter (E) permission and crosses protection domain boundaries with stack frame allocation, capability list switching, and full mLoad validation; said distinction corresponding to Church's lambda calculus application within a domain versus service invocation across domains.

Claim 5 — Machine-Status Fast Path for LAMBDA Return

The architecture of Claim 3, wherein the LAMBDA instruction stores the return address and LAMBDA-active flag in dedicated machine status registers rather than on the capability stack; and wherein the RETURN instruction, upon detecting the LAMBDA-active flag in machine status, restores the program counter from the machine status register and clears the LAMBDA-active flag, completing the return with zero stack access; said machine-status fast path providing the common-case execution path for LAMBDA invocations where no CALL or CHANGE instruction intervenes during the LAMBDA body.

Claim 6 — Self-Describing Stack Frames with 1-Bit Tag

The architecture of Claims 3 and 4, wherein every frame on the capability stack carries a 1-bit tag identifying the frame as either a CALL frame (tag = 0) or a LAMBDA frame (tag = 1); and wherein the RETURN instruction, when popping a frame from the stack, inspects the tag to determine the restoration path — performing full domain restoration for CALL frames, and performing simple program counter restoration for LAMBDA frames; said tag making the thread's execution history self-describing.

Claim 7 — Non-Nestable LAMBDA with CALL-Mediated Nesting

The architecture of Claims 3 and 4, wherein a second LAMBDA instruction executed while the LAMBDA-active flag is set in machine status causes a hardware fault, preventing uncontrolled nesting; and wherein a CALL instruction executed during a LAMBDA body saves the LAMBDA machine status as part of the CALL stack frame and clears the LAMBDA-active flag in machine status; thereby permitting a nested LAMBDA instruction within the called procedure, with the outer LAMBDA state preserved on the stack and restored when the CALL returns.

Claim 8 — Clean CR/DR Separation

The architecture of Claim 1, wherein capability registers hold exclusively Golden Tokens and data registers hold exclusively numeric values; wherein no instruction transfers raw numeric values into capability registers or extracts raw bit patterns from capability registers into data registers; and wherein the LAMBDA instruction bridges the two domains by using a capability (GT with X permission in a CR) to execute code that operates on values (arguments and results in data registers), without writing to capability registers during execution.

Claim 9 — Dual-Gate Trusted Security Base (mLoad + mSave)

The architecture of Claim 1, comprising a Trusted Security Base consisting of exactly two hardware gates:

(a) an mLoad read gate that validates every read-side capability operation through a sequential pipeline of permission check, bounds check, version match, MAC/seal validation, G-bit reset, capability register write, and thread table shadow update; wherein mLoad is the sole path for all capability register writes involving namespace dereferencing; and wherein mLoad enforces a permission gate table mapping R→DREAD, W→DWRITE, X→LAMBDA, L→LOAD, S→SAVE (c-list access), E→CALL;

(b) an mSave write gate that validates every write of a Golden Token to a C-List through a sequential pipeline of source GT version match, source GT seal validation, target C-List bounds check (verifying the destination slot index is within the C-List's allocated range), B-bit check (B=1 required for bindability), F-bit detection on target slot (routing to HTTP/tunnel for foreign objects), seal recomputation, G-bit reset, and commit; wherein mSave is the sole path for all capability writes to C-Lists;

wherein the dual-gate TSB provides symmetric validation on both read and write paths, and the total TSB comprises fewer than 400 lines of synthesizable hardware description language; and wherein any failure at any step in either gate routes to a single hardware FAULT handler with no partial state, no silent failure, and no fallback path.

Claim 10 — B (Bind) Bit for Capability Propagation Control

The architecture of Claims 1 and 9, further comprising a B (Bind) bit stored as namespace entry metadata (word1, bit 31) that controls capability propagation; wherein:

(a) B defaults to 0 on all namespace entries, meaning capabilities are non-bindable by default and cannot be saved to another C-List;

(b) the mSave write gate checks B=1 on the source GT before committing a write to a C-List, and FAULTs if B=0;

(c) CALL auto-clears B on all preserved capability registers passed to the callee, preventing the callee from propagating the caller's capabilities to other domains;

(d) Allow Bind is the explicit special case, requiring TPERM with B modifier (e.g., TPERM CR0, EB) before CALL to set B=1 on a specific capability;

thereby providing hardware-enforced control over capability distribution across protection domains, with the secure default being non-propagation.

Claim 11 — Network-Transparent RPC via Namespace-Stored Tunnel Key and F-bit Routing

The architecture of Claims 1 and 9, wherein a cryptographic key for a point-to-point RPC tunnel between two Meta Machines is stored in a standard namespace entry, accessed via CAP.LOAD with Read (R) permission through the mLoad validation path; wherein the F (Far/Foreign) flag on namespace entries marks remote resources; wherein on the read path, mLoad detects F=1 and routes the access through HTTPS fetch (for R permission) or RPC tunnel (for E permission); wherein on the write path, mSave detects F=1 on the target slot and routes the write through HTTPS flush (for W permission) or RPC tunnel; and wherein garbage collection of the tunnel key's namespace entry (version bump) instantly revokes the tunnel by invalidating all copies of the GT that references the key entry.

Claim 12 — Deterministic Garbage Collection with Bidirectional G-bit

The architecture of Claims 1 and 9, wherein deterministic garbage collection comprises a four-phase cycle (Scan-Identify-Clear-Flip) with a bidirectional G-bit mechanism; wherein:

(a) the G-bit is reset (cleared to 0) on every namespace access through both the mLoad read gate and the mSave write gate, ensuring that reachability determines liveness regardless of whether the access is a read or write operation;

(b) the Scan phase walks the reachability tree from all live roots (capability registers, call stack frames, thread table entries), clearing G=0 on reachable entries;

(c) entries still marked G=1 after scanning are unreachable and have their version bumped, instantly invalidating all outstanding Golden Tokens that reference the old version;

(d) garbage collection is implemented as a safe Turing abstraction — an atomic Turing machine hidden behind a Church-callable namespace entry, entered via CALL and exited via RETURN;

thereby preventing use-after-free vulnerabilities through deterministic version-based invalidation with zero runtime overhead on the fast path.

Claim 13 — M Permission as Transient Microcode Elevation

The architecture of Claim 1, further comprising a seventh permission bit M (Meta/Microcode) that exists only transiently on capability registers during microcode execution and is never stored in Golden Tokens; wherein the microcode elevates M on a CR to perform privileged actions including namespace reads (LOAD), namespace writes (SAVE), thread state updates (CHANGE), and garbage collection scanning; wherein M is

cleared when the microcode operation completes; and wherein no user instruction can set, test, or observe M; thereby providing privileged microcode access without requiring a privileged hardware mode, supervisor state, or kernel trap mechanism.

Claim 14 — Atomic Abstraction Architecture with Zero-OS Security

The architecture of Claims 1, 3, and 9, wherein the processor operates without a central operating system, without virtual memory, without privileged hardware modes, and without a superuser identity; wherein all system services are atomic abstractions accessed exclusively through Golden Tokens; wherein the dual-gate TSB (mLoad + mSave) provides the sole trusted path for all capability operations; and wherein the architecture achieves seven security zeros: zero OS required, zero virtual memory, zero privilege escalation, zero superuser, zero unauthorized code execution, zero unauthorized data access, and zero containment escape.

Claim 15 — Five-Phase Hardware Boot with NULL Initialization

The architecture of Claims 1 and 2, wherein the processor boots through a five-phase hardware sequence: (0) IDLE; (1) FAULT_RST, clearing all CRs to NULL type, all DRs to zero; (2) LOAD_NS, loading the namespace GT into CR15 from a hardwired bootstrap source; (3) INIT_THRD, initializing CR8 and CR5; (4) LOAD_NUC, loading CR14 and CR6; (5) COMPLETE, beginning instruction fetch; wherein Phase 1 sets all capability registers to NULL type, and Phases 2-4 load valid GTs through mLoad.

Claim 16 — Mint as Domain-Pure Namespace Method

The architecture of Claim 1, wherein the operation to create new Golden Tokens (Mint) is a method of the Namespace abstraction accessed through a chain of abstraction nesting hidden behind capability boundaries; wherein the Mint operation enforces domain purity by requiring that access rights be either Turing-domain (any combination of R, W, X) or Church-domain (any combination of L, S, E), never both; and wherein the B-bit on newly minted capabilities defaults to 0 (non-bindable).

Claim 17 — Pure Church Lambda Processor

A processor architecture comprising:

(a) a software-accessible instruction set consisting exclusively of lambda calculus reduction operations: LOAD (L permission), SAVE (S permission), CALL (E permission), RETURN, LAMBDA (X permission), and TPERM (permission verification);

(b) wherein no arithmetic instruction, no logic instruction, no comparison instruction, no branch instruction, no direct memory addressing instruction, and no register transfer instruction is available to software;

(c) wherein all arithmetic computation is performed through Church-encoded lambda calculus reductions: Church numerals for natural numbers, Church booleans for

conditional logic, Church pairs for structured data, and Y-combinator for recursive computation;

(d) wherein every instruction operates exclusively through Golden Tokens with hardware-verified permissions, validated by the dual-gate TSB of Claim 9, and every failure routes to a single FAULT handler.

Claim 18 — Security Enforcement Through Architectural Instruction Exclusion

The processor of Claim 17, wherein the architectural exclusion of Turing-domain instructions from the software instruction set eliminates, by construction rather than by mitigation:

(a) buffer overflow attacks, because no instruction can write to a computed arbitrary address;

(b) return-oriented programming attacks, because no branch instruction exists to chain code gadgets;

(c) code injection attacks, because no instruction can write executable code (domain purity prevents S and X permissions on the same GT) and no instruction can branch to an arbitrary address;

(d) privilege escalation, because no operating system, privilege rings, or superuser identity exists and no instruction can forge or modify a Golden Token;

(e) use-after-free exploits, because TPERM verifies capability validity before every operation and revoked capabilities cause immediate FAULT.

Claim 19 — Hardware I/O Mediator for Pure Lambda Processor

The processor of Claim 17, further comprising a hardware I/O mediator module that:

(a) is the sole hardware interface between pure lambda software and physical devices;

(b) intercepts SAVE instructions targeting Golden Tokens with the F (Far) flag set, wherein the namespace entry identifies a physical device class;

(c) translates Church-encoded output values into physical bus transactions;

(d) intercepts LOAD instructions on device-class Golden Tokens and returns hardware status as Church-encoded values;

(e) enforces Golden Token permissions on all device access through the mLoad validation path;

(f) is architecturally equivalent to a single trusted gate for the physical world.

Claim 20 — Church Numeral Method-Selector Dispatch

The processor of Claim 17, further comprising a method-selector dispatch mechanism wherein:

(a) a method selector value is converted to a Church numeral by applying the Church successor function DR1 times to Church zero using the LAMBDA instruction;

(b) the resulting Church numeral indexes into the abstraction's C-List to obtain the corresponding method GT;

(c) the method GT is verified via TPERM and applied via LAMBDA;

(d) no branch instruction, jump table, computed goto, or conditional logic instruction is used;

thereby implementing polymorphic method dispatch entirely through lambda calculus.

Claim 21 — Church-Encoded Arithmetic via Capability Tokens

The processor of Claim 17, wherein arithmetic operations including at least addition, subtraction, multiplication, division, modular arithmetic, exponentiation, logarithm, and square root are performed exclusively through:

(a) Church numeral encoding;

(b) capability-mediated function application, wherein each arithmetic primitive is a Golden Token in the Lambda abstraction's C-List;

(c) recursive operations using the Y-combinator Golden Token with Church LEQ for termination and Church SUCC/PRED for iteration;

(d) composite operations expressed by composing Church primitives through sequential LAMBDA applications.

Claim 22 — Three-Block Pure Church Processor Architecture

The processor of Claim 17, comprising exactly three hardware functional blocks:

(a) a Lambda Reducer that executes the six Church-domain instructions and contains no arithmetic logic unit, no barrel shifter, no condition flag register, and no branch prediction unit;

(b) a Capability Validator implementing the dual-gate TSB of Claim 9;

(c) an I/O Mediator of Claim 19;

wherein the total processor comprises fewer functional units than a conventional processor, resulting in smaller silicon area, lower power consumption, and a design amenable to formal verification.

Claim 23 — Interactive Pure Church Programming Model

A method of programming the pure Church lambda processor of Claim 17, maintaining the security properties of Claim 18, comprising:

(a) an interactive execution environment (REPL) wherein each expression is parsed, translated to capability-secured Church-domain operations, and executed through the complete capability-checked pipeline;

(b) named variable bindings following the step-by-step named-result programming style first described by Ada Lovelace in Note G (1843);

(c) program file execution with persisting variable scope;

(d) fail-safe error handling, wherein undefined variable references produce explicit errors and Turing-domain instructions produce immediate FAULT;

(e) wherein the programming model demonstrates general-purpose interactive programming with the security properties of Claim 18.

Claim 24 — Safe Turing Abstractions with Church Interface

The architecture of Claims 1 and 9, wherein Turing-domain implementations are encapsulated inside Church-callable namespace entries; wherein:

(a) the abstraction is entered only via CALL with valid E-permission Golden Token or LAMBDA with valid X-permission Golden Token;

(b) internal Turing instructions (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR) execute atomically within the abstraction, invisible to the caller;

(c) the abstraction is exited only via RETURN;

(d) the caller sees only a Church interface — CALL(Abstraction.Method(args)) — and cannot observe or access the internal Turing implementation;

thereby providing Church-domain security properties for the interface while enabling Turing-domain computational efficiency for the implementation.

Claim 25 — DATA Objects Bridging Church and Turing Domains

The architecture of Claims 1 and 9, further comprising DATA objects that are namespace entries accessed via Turing-domain DREAD and DWRITE instructions; wherein:

(a) DREAD requires R (Read) permission on the Golden Token, validated through mLoad;

(b) DWRITE requires W (Write) permission on the Golden Token, validated through mLoad;

(c) bounds validation ensures all reads and writes fall within the namespace entry's Location..Limit range;

(d) the Golden Token providing access is a Church-domain capability, while the data

operations are Turing-domain instructions;

thereby bridging Church and Turing domains through capability-mediated data access with hardware-enforced permission and bounds checking.

Claim 26 — Unified Address Space Under Capability Protection

The architecture of Claims 1 and 9, wherein memory, attached devices, and machine registers occupy segments of a single flat address space; wherein:

(a) memory addresses occupy MSB range 0x00-0xFD;

(b) attached device registers occupy MSB 0xFE;

(c) machine register bank occupies MSB 0xFF;

(d) every segment is protected by the same Golden Token validation through the mLoad read gate;

(e) without a valid Golden Token with appropriate permissions, any address range is unreachable;

thereby eliminating the need for separate I/O instructions, I/O privilege levels, or memory-mapped I/O permission mechanisms.

Claim 27 — Three Dispatch Styles for Abstraction Method Resolution

The architecture of Claims 3 and 4, wherein an abstraction's nucleus code may resolve method calls using any of three dispatch styles, selected by the abstraction's creator and invisible to the caller:

(a) a symbolic resolver style providing maximum isolation;

(b) a LAMBDA fast-path style using the LAMBDA instruction with X permission and zero stack access;

(c) a traditional compiled binary style;

wherein all three styles present the same interface to the caller, and the caller cannot determine which dispatch style is used; and wherein Style (b) is made possible exclusively by the LAMBDA instruction of Claim 3.

Claim 28 — LAMBDA as Macro-Like Code Reuse

The architecture of Claim 3, wherein the LAMBDA instruction enables a code body stored once in memory to be invoked from multiple call sites with near-zero overhead per invocation and without code duplication; wherein each invocation uses the machine-status fast path (zero stack access) when no CALL or CHANGE intervenes; and wherein the code body receives arguments and returns results through data registers, operating as a reusable function that achieves the performance of an inline macro without replicating the

code base.

ABSTRACT

A processor architecture, the Church-Turing Meta-Machine (CTMM), enforcing capability-based security through a dual-gate Trusted Security Base (TSB) comprising an mLoad read gate and an mSave write gate. Every Golden Token (GT) contains a 2-bit Type field (Inform, Outform, NULL, Abstract) and 6 permission bits organized into mutually exclusive Turing (R, W, X) and Church (L, S, E) domains. mLoad validates every read-side capability operation through permission, bounds, version, MAC, and G-bit checks; mSave validates every write of a capability to a C-List through version, seal, target bounds, B-bit (bind), and F-bit (far/foreign) checks. The B (Bind) bit, defaulting to 0, provides hardware-enforced control over capability propagation — CALL auto-clears B on preserved capabilities, and explicit TPERM is required to allow bind. A LAMBDA instruction provides lightweight in-scope code application with machine-status fast path and zero stack access. Self-describing stack frames with a 1-bit tag distinguish CALL from LAMBDA frames. The architecture eliminates the OS, virtual memory, privilege rings, and superuser, replacing them with atomic abstractions and 7 security zeros. Deterministic PP250 garbage collection uses bidirectional G-bit integrated into both mLoad and mSave. In its Pure Church variant, the processor operates with only 6 Church-domain instructions, architecturally excluding all Turing-domain instructions to eliminate buffer overflows, ROP attacks, code injection, and privilege escalation by construction. Three software proofs (HP-35 calculator, SlideRule engine, interactive REPL) and synthesizable FPGA implementations (Amaranth HDL, SystemVerilog) demonstrate computational completeness and practical realizability. The total TSB is fewer than 400 lines of synthesizable HDL — five orders of magnitude smaller than Linux, two orders of magnitude smaller than seL4.

FIGURES (Proposed)

Figure 1: Dual-Gate Trusted Security Base Architecture

Block diagram showing the two gates (mLoad and mSave) as the complete TSB. mLoad on the read path with its validation pipeline (permission → bounds → version → MAC → G-bit → CR write → thread shadow). mSave on the write path with its validation pipeline (version → seal → target bounds → B-bit → F-bit → seal recompute → G-bit → commit). Single FAULT handler receiving failures from both gates.

Figure 2: GT Format and Type Field

Bit layout of all four GT types side by side: Inform (Version|Index|Permissions|00), Outform (Version|Index|Permissions|01), NULL (Type=10), Abstract (Type=11). Shows

both Sim-32 (32-bit) and Sim-64 (64-bit) formats.

Figure 3: B-bit Capability Propagation Control

Flow diagram showing: (1) Default B=0 on new capabilities. (2) CALL auto-clearing B on preserved CRs. (3) TPERM setting B=1 for explicit bind permission. (4) mSave checking B=1 before committing write. (5) FAULT on attempted save with B=0.

Figure 4: LAMBDA vs. CALL Execution Paths

Side-by-side comparison of LAMBDA (~3 cycles: X permission verify, save PC to machine status, branch, fast-path RETURN) and CALL (10+ cycles: stack frame push, C-List switch, mLoad validation, B-bit auto-clear, domain crossing, RETURN with revalidation).

Figure 5: Machine-Status Fast Path

Flow diagram showing LAMBDA entry and RETURN behavior with zero stack access in the common case.

Figure 6: Self-Describing Stack Frames

Capability stack with interleaved CALL frames (tag=0) and LAMBDA frames (tag=1), with RETURN inspecting tag for restoration path.

Figure 7: PP250 Deterministic Garbage Collection

Four-phase cycle diagram: Scan (walk roots, clear G via mLoad/mSave) → Identify (G=1 entries unreachable) → Clear (reclaim) → Flip (bump version). Shows bidirectional G-bit integration through both mLoad and mSave.

Figure 8: Pure Church Processor Block Diagram

Three-block architecture: Lambda Reducer (6 instructions, no ALU), Capability Validator (dual-gate TSB), I/O Mediator (sole physical interface). Annotated with what is absent: no ALU, no barrel shifter, no condition flags, no branch unit.

Figure 9: Vulnerability Elimination by Construction

Table showing each vulnerability class (buffer overflow, ROP, code injection, privilege escalation, use-after-free, confused deputy) mapped to the missing instruction category and the dual-gate validation that prevents it.

Figure 10: Atomic Abstraction Architecture — 7 Zeroes

Conventional architecture (OS, VM, privilege rings, superuser — four attack surfaces) contrasted with CTMM (atomic abstractions, namespace entries, dual-gate TSB, Golden

Tokens — zero attack surfaces).

Figure 11: Safe Turing Abstractions

Diagram showing Church interface (CALL/RETURN) wrapping hidden Turing implementation (DREAD, DWRITE, IADD, etc.). Church is the armor, Turing is the sword. Caller sees only Church; internal Turing is atomic and invisible.

Figure 12: Unified Address Space

Address space diagram showing memory (0x00-0xFD), attached devices (0xFE), machine registers (0xFF), all gated by mLoad with Golden Token validation.

Figure 13: Network Transparency via F-bit and Tunnel Keys

Two Meta Machines with matching namespace entries, connected by encrypted tunnel. Shows F-bit detection in mSave routing to HTTP/tunnel, and GC version bump revoking the tunnel.

Figure 14: Three Dispatch Styles

Three-column comparison showing same caller interface resolved via symbolic resolver, LAMBDA fast-path, and traditional compiled binary.

Figure 15: Five-Phase Boot Sequence

State machine: IDLE → FAULT_RST (all CRs to NULL) → LOAD_NS → INIT_THRD → LOAD_NUC → COMPLETE.

Figure 16: Church Numeral Method Dispatch

Flow showing DR1 → Church numeral conversion via SUCC/ZERO → C-List indexing → TPERM → LAMBDA — zero branch instructions.

Figure 17: DATA Objects — Church/Turing Bridge

Diagram showing Church-domain GT providing access, Turing-domain DREAD/DWRITE performing operations, mLoad validating R/W permissions, bounds checking on Location..Limit.

ADDENDUM A: UNIVERSAL COMPUTATION TARGET (Filed March 2026)

Church-Turing Meta-Machine: Language-Independent Capability-Secured Instruction Set with Compiler-Verified Resident Object Model and Upload-Driven Abstraction Lifecycle

---

Inventor: Kenneth James Hamer-Hodges

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Lambda Calculus with the LAMBDA Instruction, NULL Capability Type, and Atomic Abstraction Architecture (Filed February 12, 2026)

Related Applications:

- Pure Church Lambda Processor: Architectural Exclusion of Turing-Domain Instructions as a Security Enforcement Mechanism (Filed February 2026)
- Church-Turing Meta-Machine: Dual-Gate Trusted Security Base with Hardware-Enforced Lambda Calculus, Deterministic Garbage Collection, and Architectural Vulnerability Elimination (Filed February 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; Compiler Architecture; Multi-Language Compilation; Upload-Driven Software Lifecycle; Scale-Free Security

---

TITLE OF THE INVENTION

Language-Independent Capability-Secured Instruction Set Architecture with Multi-Language Compiler, Resident Object Model, and Upload-Driven Abstraction Lifecycle in a Capability-Based Processor

---

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation-in-part of the CTMM patent applications filed February 2026, which disclose: the Golden Token (GT) capability architecture with domain purity enforcement separating Turing-domain (R, W, X) from Church-domain (L, S, E) permissions; the LAMBDA instruction for lightweight in-scope code application; the atomic abstraction architecture; the dual-gate Trusted Security Base (mLoad + mSave); and the Pure Church variant demonstrating that Turing-domain instructions can be architecturally excluded without loss of computational completeness.

The present application extends those disclosures by demonstrating that the capability-secured instruction set constitutes a universal computation target — a fixed, language-independent instruction set to which multiple programming paradigms (imperative, functional, declarative) can be compiled while preserving the hardware

security guarantees of the parent architecture. The invention introduces: (1) a multi-language compiler with paradigm-specific front-ends and a shared capability-aware back-end; (2) a Resident Object Model that maps source-language capability references to hardware c-list offsets; (3) a single-lump abstraction model where one namespace entry describes both code and capability list, split by hardware at CALL time; (4) an upload-driven abstraction lifecycle managed by a sole namespace authority (Navana); and (5) the architectural separation of correctness (compiler domain) from security (hardware domain).

FIELD OF THE INVENTION

The present invention relates to a compilation system and abstraction lifecycle for a capability-secured processor, wherein source programs written in multiple programming languages — including but not limited to imperative languages (JavaScript, C-like subsets) and functional languages (Haskell, lambda calculus) — are compiled to a fixed set of 20 capability-secured instructions, producing self-describing upload objects that are validated and installed by a sole namespace authority through a uniform upload protocol.

BACKGROUND

The Language-Architecture Coupling Problem

All known processor instruction sets are designed for, or evolve toward, a single programming paradigm. x86 and ARM are optimized for imperative C/C++ compilation. LISP machines (Symbolics, MIT) were designed for functional LISP. The Reduceron and GRIP were designed for Haskell-style graph reduction. Java processors (picoJava) targeted Java bytecode. In every case, the instruction set is coupled to the source language — the ISA is a hardware implementation of one language's computational model.

This coupling has three consequences:

1. Security is language-dependent. Safety guarantees available in one language (Haskell's type safety, Rust's ownership) are absent when the same hardware runs a different language (C, assembly). The hardware provides no language-independent security floor.

2. Capability systems are bolted on. CHERI, for example, adds capability checks to an existing ISA (MIPS, RISC-V, ARM). The capability mechanism is an addition to the instruction set, not a property of it. A non-capability instruction can still execute and bypass the protection model unless the compiler cooperates.

3. Compilation is trusted. The compiler is inside the trusted computing base — a compiler bug can produce code that violates the security model. This is because the

hardware relies on the compiler to emit correct instructions, and the instruction set does not architecturally prevent misuse.

The Discovery: Language-Independent Capability Target

The parent CTMM application discloses 20 instructions divided into two domains: 10 Church-domain (LOAD, SAVE, CALL, RETURN, CHANGE, SWITCH, TPERM, LAMBDA, ELOADCALL, XLOADLAMBDA) and 10 Turing-domain (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR), plus a shared RETURN. Domain purity is enforced by hardware: a GT cannot hold permissions from both domains simultaneously.

The present invention recognizes that this 20-instruction set is not merely "sufficient" for computation — it is a universal computation target in the following precise sense: source programs from fundamentally different programming paradigms (imperative and functional) compile to the same instruction set, producing semantically equivalent machine code, while the hardware security model (capability validation, domain purity, bounds checking) applies identically regardless of source language.

This has been demonstrated by reduction to practice: the CLOOMC++ compiler compiles both JavaScript (imperative, C-like syntax, explicit control flow) and Haskell (functional, lambda calculus, pattern matching) to the same 20 instructions, producing identical upload objects processed by the same namespace authority. The instruction IADD DR4, DR0, #1 is emitted for both result = who + 1 (JavaScript) and successor(n) = n + 1 (Haskell). The hardware cannot distinguish the source language — and does not need to.

The Compiler-Security Separation

A critical insight of the present invention is the architectural separation of correctness from security:

- Correctness is the compiler's responsibility. A compiler bug produces wrong answers — incorrect register allocation, wrong branch targets, miscomputed values. These are functional defects.
- Security is the hardware's responsibility. The capability model constrains from below. A compiler bug cannot: forge a Golden Token; escape the lump bounds enforced by the namespace entry; access a capability not present in the c-list; cross domain boundaries (Church ↔ Turing); or bypass the mLoad/mSave validation pipeline.

This separation means the compiler is outside the trusted computing base. No compiler bug, in any source language, can produce a security violation. The hardware enforces the invariants regardless of what code the compiler generates.

DETAILED DESCRIPTION

The 20-Instruction Universal Target

The Church Machine instruction set comprises 20 instructions in two domains:

Domain	Instruction	Encoding	Function
Church	LOAD	00000	Load GT from c-list (CR6) into capability register; req
Church	SAVE	00001	Save GT from capability register to c-list; requires S p
Church	CALL	00010	Enter abstraction scope via E-GT; splits lump into CR
Church	RETURN	00011	Exit abstraction scope; restore caller context
Church	CHANGE	00100	Modify GT permissions (reduce only)
Church	SWITCH	00101	Context switch between threads
Church	TPERM	00110	Test GT permissions; FAULT on failure
Church	LAMBDA	00111	Apply function in-scope via X permission
Church	ELOADCALL	01000	Fused LOAD+CALL (optimization)
Church	XLOADLAMBDA	01001	Fused LOAD+LAMBDA (optimization)
Turing	DREAD	01010	Read data word from DATA object via R permission
Turing	DWRITE	01011	Write data word to DATA object via W permission
Turing	BFEXT	01100	Extract bitfield from data register
Turing	BFINS	01101	Insert bitfield into data register
Turing	MCMP	01110	Compare two data registers; set condition flags
Turing	IADD	01111	Integer addition
Turing	ISUB	10000	Integer subtraction
Turing	BRANCH	10001	Conditional branch (ARM-style condition codes)
Turing	SHL	10010	Shift left
Turing	SHR	10011	Shift right

All instructions share a uniform 32-bit encoding: opcode[5] | cond[4] | dst[4] | src[4] | imm[15].

ARM-style condition codes (EQ, NE, CS, CC, MI, PL, VS, VC, HI, LS, GE, LT, GT, LE, AL, NV) apply to every instruction, providing conditional execution without separate branch-and-execute sequences.

The Multi-Language Compiler (CLOOMC++)

The CLOOMC++ compiler is a multi-front-end, single-back-end compiler targeting the 20-instruction Church Machine ISA. Each front-end parses a different source language; all front-ends share the same Resident Object Model and code generator.

Front-End Architecture

Language Detection: The compiler auto-detects the source language by examining syntactic markers. The = sign after method parameter lists, -- comment markers, and if...then...else syntax indicate Haskell. Curly braces, var declarations, and // comments indicate JavaScript. This detection is performed before parsing begins.

JavaScript Front-End (imperative paradigm):

The JavaScript front-end parses a subset of JavaScript sufficient for system programming:

Source Construct	Church Machine Instruction(s)
var x = expr	Register allocation + expression code
x = expr	Expression evaluation → target register
x + y	IADD DRz, DRx, DRy
x - y	ISUB DRz, DRx, DRy
x << n	SHL DRz, DRx, #n
x >> n	SHR DRz, DRx, #n
if (x == y) {...}	MCMP DRx, DRy + BRANCH.EQ
while (cond) {...}	Label + MCMP + BRANCH (loop)
call(Abs.Method(args))	LOAD CR from c-list + CALL
read(cr, offset)	DREAD DRx, CRy, #offset
write(cr, offset, val)	DWRITE DRx, CRy, #offset
bfext(val, pos, width)	BFEXT DRz, DRx, #pos, #width
bfins(dst, val, pos, width)	BFINS DRz, DRx, #pos, #width
return(val)	Move result to DR1 + RETURN

Haskell Front-End (functional paradigm):

The Haskell front-end parses a subset of Haskell supporting lambda calculus, pattern matching, and algebraic data:

Source Construct	Church Machine Instruction(s)
\x -> body	LAMBDA (Church function application)
f x	XLOADLAMBDA or CALL (function application)
let x = expr in body	Register binding + expression + body code
case x of { 0 -> a, 1 -> b, _ -> c }	MCMP DRx, #0 + BRANCH.EQ + MCMP DRx, #1 + BRANCH.EQ + def...
if p then a else b	MCMP DRp, #0 + BRANCH.EQ (to else) + then-code + BRANCH (...
x + y	IADD DRz, DRx, DRy
x * y	Iterative IADD loop (MCMP + BRANCH + IADD)
(a, b)	SHL DRa, #16 + BFINS DRb (pair encoding)
fst p	SHR DRp, #16 (first element extraction)
snd p	BFEXT DRp, #0, #16 (second element extraction)
succ n	IADD DRn, DRn, #1
pred n	ISUB DRn, DRn, #1

Critical observation: Both front-ends produce the same instruction for equivalent operations. result = who + 1 (JavaScript) and successor(n) = n + 1 (Haskell) both emit IADD DR4, DR0, #1. The hardware cannot distinguish the source language. This is the defining property of a universal computation target.

The Resident Object Model

The Resident Object Model (ROM) is the compiler's representation of the abstraction's capability environment. It serves as the bridge between source-language names and hardware c-list offsets.

Key insight: The c-list IS the compiler's symbol table for external references. When source code references an external abstraction (e.g., call(Memory.Allocate(size)) in JavaScript or Memory.allocate size in Haskell), the compiler must resolve this to a LOAD instruction targeting a specific c-list slot in CR6. The ROM maps:

```
Source name "Memory"  →  c-list offset 0  →  LOAD CR_temp, CR6, #0
Source name "Mint"    →  c-list offset 1  →  LOAD CR_temp, CR6, #1
```

The ROM is generated directly from the upload's capabilities array — the same source of truth that the namespace authority (Navana) uses to populate the physical c-list at installation time. The compiler never guesses offsets; it reads them from the capability declaration.

Risk mitigation (R005): If the ROM were generated independently of the capabilities array, a mismatch could cause the code to LOAD the wrong GT and CALL the wrong abstraction — capability confusion. By deriving both from the same source, this class of error is eliminated by construction.

Calling Convention

The compiler enforces a fixed register convention across all source languages:

Registers	Role	Saved By
DR0	Hardwired zero	—
DR1-DR3	Arguments and return values	Caller
DR4-DR11	Local variables	Callee
DR12-DR15	Temporaries (compiler scratch)	Caller

This convention is architectural, not advisory. The compiler allocates registers within these ranges regardless of source language. A JavaScript method and a Haskell method use the same register protocol, enabling cross-language CALL sequences — a JavaScript abstraction can CALL a Haskell-compiled abstraction through the standard E-GT mechanism with no adaptation layer.

The Single-Lump Abstraction Model

The present invention introduces a single-lump model for abstractions: one namespace (NS) entry describes one contiguous memory allocation (the "lump") that contains both code and capability list. The CALL instruction splits the lump into two domain-pure regions at runtime.

NS Entry Word1 Layout

```
Bit 31:    B (Bind) — can this GT be saved to another c-list?
Bit 30:    F (Far) — is this a remote/network resource?
Bit 29:    G (GC) — garbage collection liveness flag
Bit 28:    Chain — reserved for linked structures
Bits 27:26: Type — 00=NULL, 01=Inform, 10=Outform, 11=Abstract
Bits 25:17: clistCount — number of c-list entries (0-511)
Bits 16:0:  Limit — size of allocation (17 bits, max 131072 words)
```

clistCount is the architectural key. When clistCount > 0, the CALL instruction knows this is an abstraction with a capability list, and splits accordingly. When clistCount = 0, the NS entry describes a plain data object (no c-list, no code/data split).

CALL Lump Split

When the CALL instruction processes an Inform E-GT with clistCount > 0:

1. mLoad validation: GT type check → version match → seal verify → bounds check → E-permission check → F-bit check → deliver NS entry
2. Parse word1: Extract clistCount and limit from the NS entry
3. Compute split point: clistStart = (limit + 1) - clistCount
4. Create CR14 (Code Region): location = base, limit = clistStart - 1, permissions = X-only (hardcoded)
5. Create CR6 (C-List Region): location = base + clistStart, limit = clistCount - 1, permissions = L-only (hardcoded)
6. Set PC = 0: Execution begins at the first instruction of the code region

Critical security invariant (R001): CR14 permissions are hardcoded to X-only (execute). CR6 permissions are hardcoded to L-only (load capability). These permissions are not derived from the E-GT or the NS entry — they are architectural constants enforced by the CALL instruction. This prevents:

- Code reading its own capabilities as data (CR14 cannot Load)
- C-list entries being executed as code (CR6 cannot eXecute)
- Any cross-domain contamination between code and capability regions

Lump Memory Layout

```
┌──────────────────────────────────────┐ offset 0
│ Method table + compiled code         │ → CR14 (Turing domain, X-only)
│ (produced by [CLOOMC](https://sipantic.blogspot.com/2025/03/xx.html)++ compiler)      │
├──────────────────────────────────────┤ codeEnd
│ FREESPACE (power-of-2 padding)       │ inaccessible (beyond code limit)
├──────────────────────────────────────┤ clistStart = allocSize - clistCount
│ C-list (Golden Token slots)          │ → CR6 (Church domain, L-only)
│ (populated by Navana from upload)    │
└──────────────────────────────────────┘ allocSize (power-of-2)
```

The freespace region is architecturally inaccessible — it lies beyond CR14's limit and before CR6's base. This provides growth room for code updates without reallocating the lump.

The Upload-Driven Abstraction Lifecycle

The present invention introduces a uniform upload protocol for creating, updating, and removing abstractions. Every abstraction — whether compiled from JavaScript, Haskell, or hand-assembled — enters the system through the same upload format:

```
{
    "abstraction": "Name",
```

```
    "type": "abstraction",
    "grants": ["E"],
    "capabilities": [
        { "target": 7, "name": "Memory", "grants": ["E"] },
        { "target": 6, "name": "Mint", "grants": ["E"] }
    ],
    "methods": [
        { "name": "MethodName", "code": [0x12345678, 0x9ABCDEF0] }
    ]
}
```

Fields:

- abstraction: Problem-oriented name (human-readable)
- type: "abstraction" (code + c-list) or "namespace" (isolated child namespace)
- grants: Permissions granted to callers via the E-GT
- capabilities: C-list wiring — each entry specifies a target NS index, name, and delegated permissions
- methods: Compiled code — each method is a named sequence of 32-bit instruction words

The upload is both the IDE's output format and the runtime's input format. The compiler produces uploads; the namespace authority (Navana) consumes them.

Navana: Sole Namespace Authority

Navana is the sole writer of namespace table entries after boot. This is an architectural invariant:

1. Boot (mElevation): A single raw write installs Navana's own NS entry — the one exception to the rule
2. All subsequent NS writes: Routed through Navana.Add, which finds a free slot, writes the 3-word NS entry (location, word1 with clistCount and type, seal), and returns the NS index + version
3. Abstraction creation: Navana.Abstraction.Add processes the upload: allocates a power-of-2 lump via Memory, writes compiled code to the code region, populates the c-list with delegated GTs, creates the NS entry via Navana.Add, and forges the E-GT
4. Drop mElevation: After boot, machine-level write access is permanently relinquished — Navana operates through its own capabilities thereafter

Upload validation (R007): Navana validates every upload before installation:

- codeSize + clistCount <= allocatedSize (no overlap between code and c-list)
- Each capability target exists AND the creator holds sufficient permissions to delegate
- clistCount <= 511 (fits in 9 bits of word1)
- allocatedSize is a valid power-of-2
- Integer overflow/underflow checks on all size arithmetic

Mint delegates to Navana: The Mint abstraction (GT lifecycle management) delegates NS

entry creation to Navana.Add. Mint.Create performs domain validation and GT forging; the actual namespace write is Navana's exclusive authority.

GT Type System

The architecture defines four GT types enforced by the hardware type field (bits 1:0):

Type	Encoding	Semantics
NULL	00	Zero value, no capability. Any dereference → FAULT
Inform	01	GT points to memory via an NS entry. Used for all abstrac...
Outform	10	GT points to remote memory. F-bit set automatically. Cros...
Abstract	11	GT IS the value. No memory pointer. Constants (pi), immut...

All abstractions use Inform (01) GTs. The clistCount field in word1 distinguishes abstractions (clistCount > 0) from plain data objects (clistCount = 0).

Scale-Free Security

The abstraction model is scale-free — the same security mechanism (GT + NS entry + lump + CALL split) applies identically at every scale:

- Single child: One namespace, one set of abstractions, one parent's c-list = parental approval
- Family: Multiple sibling namespaces, each isolated, parent holds E-GTs to grant/revoke access
- School: Teacher namespace + student namespaces, Negotiate abstraction for dual-approval grants
- Organization: Hierarchical namespaces, Outform GTs for cross-boundary access, Tunnel for networking

The upload protocol, the namespace authority model, and the capability validation pipeline are identical at every scale. No "enterprise mode" or "admin privilege" exists — the same 20 instructions, the same mLoad pipeline, the same CALL split.

REDUCTION TO PRACTICE

Multi-Language Compilation Proof

The CLOOMC++ compiler has been implemented as a working system (simulator/cloomc_compiler.js) with both JavaScript and Haskell front-ends. The following abstractions have been compiled and executed:

JavaScript front-end — 7 resident objects:

1. Hello: Single method (Greet) — IADD + RETURN (3 instructions)

2. Counter: Two methods (Increment, Add) — IADD + RETURN each
3. Memory: Two methods (Allocate, Free) — DREAD + SHR + SHL + DWRITE + IADD + RETURN
4. Mint: Two methods (Create, Revoke) — LOAD + CALL + RETURN (c-list wiring via ROM)
5. Navana: Nine methods — Init, Add, Remove, Abstraction.Add/Remove/Update, Manage, Monitor, IDS
6. Salvation: Four methods — LOAD, TPERM, LAMBDA, TransitionToNavana (boot verification)
7. Scheduler/Stack/DijkstraFlag: Stub methods (RETURN-only, awaiting Phase 2 compilation)

Haskell front-end — 4 resident objects:

1. ChurchMath: Five methods — successor (IADD), add (IADD), multiply (IADD loop with MCMP+BRANCH), predecessor (MCMP+BRANCH+ISUB), isZero (MCMP+BRANCH)
2. ChurchPair: Four methods — makePair (SHL+BFINS), first (SHR), second (BFEXT), swap (SHR+BFEXT+SHL+BFINS)
3. ChurchCase: Three methods — factorial (MCMP+BRANCH+ISUB), classify (MCMP chain), abs (MCMP+BRANCH+ISUB)
4. ChurchLambda: Four methods — identity (RETURN), constant (RETURN), double_succ (IADD+IADD), letExample (IADD+IADD)

Universal target proof: The instruction IADD DR4, DR0, #1 appears in both Hello.Greet (JavaScript: result = who + 1) and ChurchMath.successor (Haskell: method successor(n) = n + 1). The hardware executes identical machine code for equivalent operations across paradigms.

Boot Sequence Proof

The boot sequence processes an upload array of 11 abstractions through the following stages:

1. FAULT_RST: Zero all registers
2. mElevation: Raw write Navana's NS entry (sole exception to Navana-only writes)
3. Process remaining uploads through Navana.Add
4. Drop mElevation permanently
5. CALL Salvation → Salvation validates security pipeline → Transition to Navana
6. Navana runs forever (no RETURN)

All Phase 1 abstractions (Memory, Mint, Navana) run real CLOOMC++-compiled code. Phase 2-4 abstractions (Scheduler, Stack, DijkstraFlag, hardware attachments, math, lambda, social, IDE, internet, GC) have stub methods (RETURN-only) until compiled.

Security Risk Register

Nine risks (R001-R009) have been identified, documented, and resolved:

Risk	Severity	Description	Status
R001	CRITICAL	CALL must hardcode CR14=X, CR6=1	RESOLVED
R002	MEDIUM	25-bit FNV seal strength	ACCEPTED (adequate for target hardware)
R003	LOW	Boot raw write single point of failure	RESOLVED
R004	HIGH/LOW	Compiler incorrect code generation	RESOLVED (security unaffected by compiler bugs)
R005	MEDIUM	C-list offset mismatch	RESOLVED (ROM derived from capabilities array)
R006	MEDIUM	Haskell closure variable capture	RESOLVED (closures capture DR only, never CR)
R007	HIGH	Upload validation integer underflow	RESOLVED (Navana validates all uploads)
R008	MEDIUM	Register spilling / calling convention	RESOLVED (fixed convention DR0-3/DR4-11/DR12-15
R009	FOUNDATIONAL	Namespace isolation guarantee	SECURE (all contingencies resolved)

End-to-End Verification

Automated end-to-end testing confirms:

- JavaScript source compiles and produces [JavaScript] tagged output with correct hex instruction words
- Haskell source compiles and produces [Haskell] tagged output with correct hex instruction words
- Both produce valid compiled abstractions processed by Navana.Abstraction.Add
- Boot sequence completes with all 11 abstractions installed
- CR14 and CR6 are correctly set from single NS entry with clistCount split

---

PROPOSED CLAIMS

Claim 24 — Universal Computation Target Instruction Set

A processor instruction set architecture comprising:

(a) a fixed set of instructions divided into two domains — a Church domain for capability operations (LOAD, SAVE, CALL, RETURN, CHANGE, SWITCH, TPERM, LAMBDA, ELOADCALL, XLOADLAMBDA) and a Turing domain for data operations (DREAD, DWRITE, BFEXT, BFINS, MCMP, IADD, ISUB, BRANCH, SHL, SHR);

(b) wherein all instructions share a uniform encoding format comprising an opcode field, a condition code field, a destination register field, a source register field, and an immediate value field;

(c) wherein the instruction set constitutes a universal computation target, demonstrated by the successful compilation of source programs from at least two fundamentally different programming paradigms — imperative and functional — to the same instruction set, producing semantically equivalent machine code for equivalent operations;

(d) wherein the hardware security model — comprising capability token validation, domain purity enforcement, bounds checking, and permission verification through a trusted gate pipeline — applies identically to all compiled code regardless of source language;

(e) wherein the compiler that translates source programs to the instruction set is architecturally outside the trusted computing base, such that no compiler bug in any source language can produce a security violation, because the hardware enforces capability invariants independent of the code generated.

Claim 25 — Multi-Language Capability Compiler with Resident Object Model

A compilation system for the processor of Claim 24, comprising:

(a) a multi-front-end compiler architecture wherein each front-end parses a different source language and all front-ends share a common back-end that generates instructions from the fixed instruction set of Claim 24;

(b) a language detection mechanism that automatically identifies the source language from syntactic markers before parsing begins;

(c) a Resident Object Model that maps source-language references to external abstractions (capability names) to hardware c-list offsets, wherein the mapping is derived directly from the upload's capability declaration — the same source of truth used by the namespace authority to populate the physical c-list at installation time;

(d) a fixed calling convention enforced across all source languages, wherein a first set of data registers is designated for argument passing and return values, a second set for callee-saved local variables, and a third set for caller-saved temporaries;

(e) wherein the Resident Object Model ensures that a capability reference in source code (e.g., call(Memory.Allocate(size)) in an imperative language or Memory.allocate size in a functional language) compiles to the same hardware instruction sequence — a LOAD from the correct c-list offset followed by a CALL — regardless of source language;

(f) wherein the compiler output is a self-describing upload object containing the abstraction name, capability declarations, and compiled method code, suitable for processing by the namespace authority of Claim 28.

Claim 26 — Single-Lump Abstraction Model with Hardware CALL Split

A method of managing abstractions in the processor of Claim 24, comprising:

(a) allocating a single contiguous memory region (lump) for each abstraction, wherein the lump contains both compiled code and a capability list (c-list);

(b) describing the lump with a single namespace entry, wherein the namespace entry's word1 field contains a clistCount value indicating the number of c-list entries;

(c) upon execution of a CALL instruction targeting the abstraction:

- computing a split point as clistStart = (limit + 1) - clistCount;
- creating a first context register (CR14) pointing to the code region with permissions hardcoded to execute-only (X);
- creating a second context register (CR6) pointing to the c-list region with permissions hardcoded to load-only (L);
- setting the program counter to zero;

(d) wherein the code region permissions (X-only) and c-list region permissions (L-only) are architectural constants enforced by the CALL instruction, not derived from the Golden Token or namespace entry permissions;

(e) wherein this hardcoded domain split prevents code from reading capabilities as data and prevents c-list entries from being executed as code, maintaining domain purity across the code/capability boundary.

Claim 27 — Capability-Type Taxonomy with Architectural Semantics

The processor of Claim 24, wherein each Golden Token contains a two-bit type field with the following architectural semantics:

(a) NULL (00): zero value, no capability; any dereference produces an immediate FAULT;

(b) Inform (01): the GT points to a memory-backed object described by a namespace entry; used for all abstractions and data objects; the clistCount field in the namespace entry distinguishes abstractions (clistCount > 0) from plain data objects (clistCount = 0);

(c) Outform (10): the GT points to a resource in a remote namespace; the Far (F) bit is set automatically; used for cross-namespace and network-transparent access;

(d) Abstract (11): the GT IS the value; no memory pointer or namespace entry is required; used for mathematical constants, immutable credentials, and escale variables;

wherein the type field is validated by the hardware trusted gate pipeline on every access, and type-specific behavior (memory dereference for Inform, network routing for Outform, immediate value for Abstract) is performed by the hardware without software intervention.

Claim 28 — Sole Namespace Authority with Upload-Driven Lifecycle

A method of managing the namespace table in the processor of Claim 24, comprising:

(a) designating a single abstraction (Navana) as the sole authority permitted to write namespace table entries after the boot sequence;

(b) during boot, performing exactly one privileged write (mElevation) to install Navana's own namespace entry, and thereafter permanently relinquishing privileged write access;

(c) routing all subsequent namespace table writes through Navana.Add, which finds a free

namespace slot, writes the three-word namespace entry (location, word1 with clistCount and type and seal), and returns the namespace index and version;

(d) processing abstraction creation through Navana.Abstraction.Add, which:

- validates the upload object (code size + c-list count within allocation bounds, capability delegation authority verified, integer overflow checks);
- allocates a power-of-2 memory lump via the Memory abstraction;
- writes compiled code to the code region of the lump;
- populates the c-list region with delegated Golden Tokens;
- creates the namespace entry via Navana.Add;
- forges an Inform E-GT (Enter permission, type=01) and returns it to the creator;

(e) wherein other abstractions (including Mint, the GT lifecycle manager) delegate namespace entry creation to Navana, ensuring a single point of namespace authority and a single validation path for all abstractions regardless of source language or creation method.

Claim 29 — Compiler-Security Architectural Separation

The compilation system of Claim 25, wherein:

(a) the compiler is architecturally outside the trusted computing base of the processor;

(b) a compiler bug can produce functionally incorrect code (wrong register allocation, incorrect branch targets, miscomputed values) but cannot produce a security violation;

(c) security invariants enforced by the hardware independent of compiler output include:

- Golden Token validation (version match, seal verification, permission check) on every memory access;
- domain purity (Church-domain and Turing-domain permissions cannot coexist on a single GT);
- bounds checking (code cannot access memory beyond the lump limits set by the namespace entry);
- CALL split hardcoding (CR14=X-only, CR6=L-only regardless of compiled code);
- c-list authority (code can only LOAD GTs present in its c-list; no GT can be forged by any instruction sequence);

(d) wherein this separation holds for all source languages processed by the compiler, such that adding a new language front-end to the compiler cannot weaken the security model — the hardware provides a language-independent security floor.

Claim 30 — Cross-Paradigm Capability Interoperability

The compilation system of Claim 25, wherein:

(a) abstractions compiled from different source languages can invoke each other through

the standard CALL mechanism, using the same E-GT protocol, register convention, and c-list wiring;

(b) a first abstraction compiled from an imperative language and a second abstraction compiled from a functional language can hold E-GTs to each other in their respective c-lists;

(c) the CALL instruction, the mLoad validation pipeline, and the lump split mechanism operate identically regardless of the source languages of the caller and callee;

(d) no adaptation layer, foreign function interface, or runtime type conversion is required for cross-paradigm invocation — the hardware protocol is the sole interface.

Claim 31 — Scale-Free Security Architecture

The processor of Claim 24, wherein:

(a) the abstraction model (GT + namespace entry + lump + CALL split) applies identically at every organizational scale — individual user, family, school, enterprise, and inter-organizational;

(b) namespace isolation is achieved by each entity having its own namespace table, memory region, and set of Golden Tokens;

(c) access across namespace boundaries requires a valid Outform GT (type=10) with the Far bit set, processed through the same mLoad validation pipeline;

(d) capability delegation across scales uses the same upload protocol and Navana validation path;

(e) revocation at any scale is achieved by incrementing the version number in the namespace entry, instantly invalidating all outstanding GT copies — with no "revocation list" propagation delay;

(f) no "admin mode," "superuser privilege," "root access," or "enterprise tier" exists — the same 20 instructions, the same mLoad pipeline, and the same CALL split apply to every user at every scale.

PRIOR ART DISTINCTION

System	Multi-Language Target	Capability Security	Compiler Outside TCB	Upload Lifecycle	Scale-Free
JVM (Sun/Oracle)	Yes (Java, Kotlin, Scala)	No	No	No	No
CLR (.NET/Microsoft)	Yes (C#, F#, VB)	No	No	No	No
LLVM	Yes (C, Rust, Swift)	No	No	No	No
CHERI (Cambridge)	No (C/C++ focus)	Yes	No	No	No
WebAssembly (W3C)	Yes (C, Rust, Go)	No (sandbox only)	No	No	No
LISP Machines	No (LISP only)	No	No	No	No

Reduceron	No (Haskell only)	No	No	No	No
CTMM (parent)	No (single language)	Yes	Partially	No	Partially
Church Machine (this CTP)	Yes	Yes	Yes	Yes	Yes

Key distinctions from JVM/CLR/LLVM: These are multi-language compilation targets, but they lack hardware capability security. The compiler is inside the trusted computing base — a compiler bug can produce code that violates the security model (buffer overflows in JVM native methods, unsafe blocks in .NET, undefined behavior in LLVM-compiled C).

Key distinction from CHERI: CHERI adds capabilities to an existing ISA but is designed primarily for C/C++ memory safety. The compiler must cooperate with the capability model. The Church Machine's ISA is inherently capability-secured — every instruction operates through GTs — making the compiler's cooperation unnecessary for security.

Key distinction from WebAssembly: Wasm is a multi-language target with a sandbox model, but sandboxing is weaker than capability security — a Wasm module has ambient authority within its sandbox. The Church Machine's c-list model provides fine-grained authority: each abstraction can only access the specific capabilities delegated to it.

FIGURES (Proposed)

Figure 24: Multi-Language Compilation to Universal Target

Diagram showing JavaScript and Haskell source code flowing through language-specific front-ends into the shared Resident Object Model and code generator, producing identical instruction words for equivalent operations, with the compiled output feeding into Navana.Abstraction.Add.

Figure 25: Single-Lump CALL Split

Diagram showing a single namespace entry pointing to a contiguous lump, with the CALL instruction splitting the lump at the clistStart boundary into CR14 (X-only code) and CR6 (L-only c-list), with freespace between.

Figure 26: Compiler-Security Separation

Diagram showing the trust boundary: compiler produces code words (correctness domain, outside TCB); hardware validates GTs, enforces bounds, hardcodes domain permissions (security domain, inside TCB). Arrows showing that compiler bugs affect correctness within the security perimeter but cannot breach it.

Figure 27: Upload-Driven Lifecycle

Diagram showing the lifecycle: source code → CLOOMC++ compiler → compiled

abstraction → Navana.Abstraction.Add (validation) → Memory.Allocate (power-of-2 lump) → code write + c-list populate → NS entry creation → E-GT forge → return to creator.

Figure 28: Scale-Free Security Model

Diagram showing identical security mechanisms (GT + NS entry + CALL split + mLoad) applied at individual, family, school, and organizational scales, with Outform GTs bridging namespace boundaries.

ABSTRACT

A capability-secured processor instruction set architecture that serves as a universal computation target for multiple programming paradigms. The fixed 20-instruction set — comprising 10 Church-domain capability operations and 10 Turing-domain data operations — accepts compiled output from imperative languages (JavaScript) and functional languages (Haskell) through a multi-front-end compiler with a shared Resident Object Model and code generator. The compiler maps source-language capability references to hardware c-list offsets, producing self-describing upload objects processed by a sole namespace authority (Navana) through a uniform validation and installation protocol. A single-lump abstraction model stores both code and capability list in one memory allocation, described by one namespace entry; the CALL instruction splits the lump into domain-pure regions (code=execute-only, c-list=load-only) using a clistCount field in the namespace entry. The compiler is architecturally outside the trusted computing base: no compiler bug in any source language can produce a security violation, because the hardware enforces capability invariants — token validation, domain purity, bounds checking, and permission verification — independent of the generated code. The architecture is scale-free: the same security mechanisms apply identically from individual users to organizations, with no privileged modes or administrative overrides.

ADDENDUM B: ABSTRACT GT I/O AND NETWORK ADDRESSING (Filed March 2026)

Church-Turing Meta-Machine: Hardware-Routed Abstract Capability Tokens for Unified I/O, Network Tunneling, and Guarantee-Based Service Scoping

---

Inventor: Kenneth James Hamer-Hodges

Filing Date: March 2026

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Capability Security Through Dual Trusted Gates, Lambda Calculus Integration, and Architectural Vulnerability Elimination (Filed February 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; I/O Virtualization; Network Security; Service-Oriented Architecture; Deterministic Garbage Collection; Domain Separation

---

TITLE OF THE INVENTION

Church-Turing Meta-Machine: Abstract Golden Token I/O and Network Addressing Architecture Enabling Hardware-Enforced Guaranteed Crime-Free Business Services Through Structural Capability Scoping by Profession, Language, Nationality, and Age

---

CROSS-REFERENCE TO RELATED APPLICATIONS

This continuation-in-part extends the CTMM patent applications filed February 2026, which disclosed:

1. The Golden Token capability architecture and dual-gate trusted security base (mLoad / mSave)
2. Domain purity enforcement separating Turing-domain (R, W, X) from Church-domain (L, S, E) permissions
3. The LAMBDA instruction for lightweight in-scope code application
4. The atomic abstraction architecture and deterministic garbage collection (G-bit mechanism)
5. The Pure Church Lambda Machine, demonstrating computational completeness through exclusive Church-domain instructions

The present application extends that disclosure with:

1. Abstract Golden Tokens — a novel capability token class (gt_type = 11_2) whose word1_location field holds a hardware-routed sentinel address instead of a namespace slot index
2. The Abstract Address Space — a 32-bit reserved range (0xFE000000–0xFFFFFFFF) controlled exclusively by the IDE for I/O peripherals, network tunnels, and system resources
3. The Home Base Tunnel (0xFF000000) — the single outbound network gateway through which all CTMM network connectivity flows, with optional programmer-defined backup IDE addresses (Word 2/3)

4. MTBF Qualification and Downloadability Regulation — hardware-tracked reliability metrics that gate whether abstractions may propagate beyond their provisioning c-list, with three tiers: Isolated (local only), User-regulated (individual distribution), Namespace-regulated (full namespace access)
5. Local Peripheral Autonomy — CTMMs identify and secure locally attached hardware (UART, GPIO, Timer, Display) without any IDE connection, enabling air-gapped and offline operation
6. Guaranteed Crime-Free Business Services — structural capability scoping (not policy-based filtering) that prevents access to out-of-scope services without any bypass path, enabling verifiable safety for professional, language-specific, jurisdictional, and age-appropriate service isolation
7. Secure Individuality of Abstractions — each abstraction's identity is its unique GT set in its c-list, eliminating the "privileged superuser" attack window and preventing confused deputy attacks entirely

---

FIELD OF THE INVENTION

The present invention relates to a processor architecture that provides a unified capability-token-based mechanism for hardware-routed I/O and remote network access, eliminating the need for separate I/O subsystems, device drivers, or network protocol stacks. The architecture enables the IDE to establish deterministic, verifiable service scoping at the architectural level — not through runtime policy enforcement, but through structural capabilities — such that a user's CTMM provisioned for a specific profession, language, jurisdiction, or age group cannot access any service outside that scope, regardless of what code runs on the machine.

BACKGROUND OF THE INVENTION

The I/O Problem

Contemporary processor architectures separate I/O from computation through a separate subsystem: an I/O controller, device drivers, and operating system I/O stacks. This separation creates multiple security boundaries:

1. I/O Controller Privilege — The I/O controller runs privileged firmware that any code on the CPU can request, creating a confused-deputy vulnerability. Malicious code can request the I/O controller to access any peripheral it manages.
2. Device Driver Complexity — Device drivers are privileged code (typically millions of lines) that perform untrusted I/O operations on behalf of applications. Buffer overflows in drivers escalate to kernel privilege.

3. Global I/O Namespace — All I/O resources are accessed by name (e.g., /dev/uart0, /dev/gpio5), shared across all processes. No isolation between applications or users.
4. Network Subsystem as Privileged Intermediary — All network access routes through a privileged TCP/IP stack. The "network" is a privileged black box that applications cannot directly control or verify.

The Service Scoping Problem

In contemporary systems, service scoping is enforced through policy layers running on top of a privileged substrate:

- Content filters (block access to non-professional content) run on the privileged network stack.
- Age gates (block adult content) run on the privileged application server.
- Jurisdiction checks (data residency) run on the privileged cloud infrastructure.
- Language routing (direct to the right service) runs in privileged DNS/load-balancer logic.

Every one of these policies is a filter applied after the capability to access the resource is granted. Filters can be bypassed by:

- VPN and proxy services (defeat jurisdiction checks)
- DNS spoofing and man-in-the-middle (defeat language routing)
- Malicious extensions and injected scripts (defeat content filters)
- Buffer overflow in the filter itself (defeat age gates)

No conventional system provides a structural guarantee that access is impossible without the right token.

The Trusted Network Gateway Problem

In contemporary architectures, the network is accessed through a single privileged gateway (the TCP/IP stack, the hypervisor's network device, the cloud provider's edge router). If this gateway is compromised — or if the entity controlling it becomes adversarial — all network access can be monitored, throttled, or redirected. There is no way for the user or the CTMM to detect or prevent this.

The Discovery

The parent CTMM application's Abstract GT type field opens a new possibility: a capability token that is not a namespace reference, but a hardware-routed sentinel address. This token type can be provisioned by the IDE exclusively at boot time — before any user code runs — and cannot be forged or synthesized by software.

By extending this principle to a full Abstract Address Space, the CTMM can:

1. Make I/O resources unforgeable capabilities — no code can access a peripheral

without holding the Abstract GT for it

2. Make network access an unforgeable capability — the Home Base tunnel (0xFF000000) is the only outbound connection; all network access flows through it
3. Enable structural service scoping — a child CTMM provisioned without the GT for adult services has no path to them, regardless of what code it runs
4. Enable local autonomy — peripherals are identified and secured during local hardware boot, not by IDE decree
5. Enable offline operation — local services (UART, GPIO, storage) work with no IDE connection; the Home Base tunnel is optional

---

THE ABSTRACT ADDRESS SPACE: A NEW RESERVED CAPABILITY DOMAIN

What Is an Abstract GT?

An Abstract Golden Token is a 128-bit capability register with:

```
Word 0 (32-bit GT):
  [15:0]  object_id / slot_id (not used for NS lookup — sub-identifier within Abstract range)
  [22:16] gt_seq (not used — zero for Abstract GTs)
  [24:23] gt_type = 11₂ (Abstract)
  [30:25] permissions (R, W, X, L, S, E)
  [31]    b_flag (may be propagated via mSave)

Word 1: word1_location (32-bit Abstract Address — the hardware-routed sentinel)
Word 2: word2_backup1 (first backup Abstract Address; 0x00000000 = not configured)
Word 3: word3_backup2 (second backup Abstract Address; 0x00000000 = not configured)
```

Unlike Inform GTs (which point to namespace entries) and Outform GTs (which point to remote resources), an Abstract GT's word1_location is not dereferenced. Instead, it is matched directly against the Abstract Address Space table in hardware. The address is the token's identity.

The Abstract Address Space Layout

The 32-bit word1_location range is reserved exclusively for the IDE:

```
0x00000000 — 0xFDFFFFFF    Real RAM — never an Abstract GT address
0xFE000000 — 0xFEFFFFFF    Local hardware peripheral range (64K entries)
                            UART, GPIO, Timer, Display, Storage identified by
                            CTMM during local hardware boot
0xFF000000                 Home Base tunnel — primary outbound network gateway
0xFF000001 — 0xFF0000FE    IDE-allocated tunnel channels (254 named remote services)
                            Each is a distinct encrypted tunnel to a named endpoint
0xFF0000FF — 0xFFFEFFFF    Reserved for future IDE-defined Abstract resources
0xFFFF0000 — 0xFFFFFFFD    Reserved for future system Abstract GTs
0xFFFFFFFE                 SWITCH PassKey for CR13 (IRQ Thread) — from Task #58
0xFFFFFFFF                 SWITCH PassKey for CR15 (Namespace) — from Task #58
```

Critical property: Abstract Addresses are not stored in the namespace table. There is no CRC validation, no namespace lookup, no lump header dereference. The address alone is the capability.

Unforgeability

Software cannot construct an Abstract GT with a reserved address because:

1. Only the IDE (running at boot, before user code) can write directly to capability registers
2. After boot, Abstract GTs can only be copied (via mSave, if b_flag=1), attenuated (via TPERM, removing permissions), or nullified
3. No instruction allows synthesis of a GT from components

A program that was not given a GT for a resource cannot obtain one — period.

THE HOME BASE TUNNEL: UNIFIED NETWORK GATEWAY

What It Is

The Home Base Tunnel (Abstract Address 0xFF000000) is an Abstract GT that represents the CTMM's outbound connection to the IDE and cloud infrastructure. It is the sole network interface through which all CTMMs communicate with the outside world.

The Home Base GT is provisioned at boot by the IDE with:

- word1_location = 0xFF000000
- perms typically = R | W | E (receive, send, RPC invoke)
- word2_backup1 and word3_backup2 = programmer-defined fallback IDE addresses (optional)

How It Works

Operations on the Home Base GT route directly to the hardware tunnel driver:

Permission	Operation	Meaning
R (Read)	DREAD / mLoad	Receive data from Home Base (IDE push, config, MTBF thres...
W (Write)	DWRITE / mSave	Send data to Home Base (telemetry, audit logs, MTBF count...
E (Enter)	CALL	Invoke a named remote service via encrypted RPC tunnel

Security

The Home Base is unforgeable:

- Only the IDE can write it at boot

- Code without the Home Base GT has no path to the network
- The b_flag controls whether the GT may be propagated (default: 0, not propagable)
- Revoking network access is instant: set the Home Base GT to NULL in the c-list

Programmer-Defined Backup IDEs (Novel)

A programmer can configure up to two backup IDE addresses in Word 2 and Word 3:

```
Primary:   word1_location = 0xFF000000 (main cloud IDE)
Backup #1: word2_backup1 = 0xFF000001 (developer's private server)
Backup #2: word3_backup2 = 0xFF000002 (team fallback)
```

Hardware implements failover: if the primary is unreachable, try Backup #1; if that fails, try Backup #2. All three are provisioned atomically at boot — user code cannot change them.

Novel advantage: A developer can ensure their CTMM falls back to a trusted private IDE, not a compromised public one, without any degradation in security. The backup addresses are unforgeable, hardware-validated, and immutable.

LOCAL PERIPHERAL SECURITY: AUTONOMOUS OPERATION

The Core Insight

Each CTMM can identify and secure locally attached equipment entirely on its own — without any IDE connection, network access, or remote authority.

During the CTMM's hardware boot sequence (before any software runs):

1. The hardware probe enumerates attached peripherals (UART, GPIO, Timer, Display, Storage)
2. For each peripheral, the boot sequence assigns it an Abstract Address from the local range (0xFE000000+)
3. The boot sequence creates an Abstract GT for each peripheral and provisions it into the appropriate c-list
4. All of this happens offline, locally, before the IDE is even contacted

Security Advantages

Scenario	Conventional System	CTMM with Local Autonomy
Air-gapped operation (no network)	I/O locked until network available	All local I/O fully functional
Offline mode	Peripherals inaccessible without cloud	Peripherals secured locally with full GT enforcement
Malicious IDE	IDE controls peripheral access; IDE can deny all I/O	IDE does not provision local peripherals; CTMM controls them
Autonomous agent in remote location	Must phone home for every I/O operation	Operates independently for all local resources

CTMM is the Authority for Its Own Hardware

The security decision — "which abstractions receive the peripheral GTs, with what permissions" — is made by the local boot policy, not by any remote party. A remote IDE has no ability to:

- Revoke access to a locally attached UART
- Deny GPIO access
- Block storage operations
- Isolate the CTMM from its own hardware

This is a fundamental shift from conventional architectures, where the OS (a privileged entity) mediates all I/O access.

MTBF QUALIFICATION AND DOWNLOADABILITY REGULATION

The Problem

When an abstraction is propagated from one CTMM to another via mSave, the receiving CTMM accepts it without question. But what if the abstraction is unreliable? What if it crashes frequently, corrupts data, or is malicious?

Contemporary systems rely on trust relationships (code signing, sandboxing, user reputation) to decide what to accept. All of these can be forged or manipulated.

The Solution: Hardware-Tracked MTBF Qualification

Every Secure Abstraction carries a hardware-tracked MTBF qualification — a reliability metric that determines whether it may be distributed beyond its local CTMM and, if so, to whom.

The hardware tracks two counters per abstraction in the namespace entry:

- invocation_count — number of times the abstraction has been called
- failure_count — number of those calls that resulted in a FAULT or exception

```
MTBF score = invocation_count / (failure_count + 1)
```

Three Downloadability Tiers

Tier	MTBF Condition	S Permission	Scope
Isolated	Below threshold or unvalidated	Hardware-locked S=0	Local CTMM only; cannot propagate
User-regulated	Meets user-tier MTBF	S enabled	Individual user distribution via Home Base tunnel
Namespace-regulated	Meets namespace-tier MTBF	S enabled	Full namespace access; CR15 validates each downloa

IDE Control of Thresholds

The parent IDE sets and updates the MTBF thresholds remotely via the Home Base tunnel:

```
Threshold payload (signed by IDE):
  isolated_floor = 0.90          — MTBF score below this locks S=0
  user_tier_threshold = 0.99     — score above this unlocks S for user distribution
  ns_tier_threshold = 0.999      — score above this unlocks namespace distribution
  min_invocations = 100          — don't qualify until at least 100 invocations
```

The IDE can raise thresholds (tighten quality requirements) or lower them without firmware updates. Thresholds take effect on the next invocation cycle.

Telemetry and Trust

The CTMM sends MTBF telemetry (counters + timescale data) back to the IDE via Home Base W permission, signed with HMAC-SHA256. The IDE maintains a permanent MTBF record per abstraction per CTMM, creating a distributed reputation system:

- An abstraction that fails on one CTMM (low MTBF score) becomes less trustworthy on all CTMMs through IDE policy update
- A highly reliable abstraction (high MTBF score) qualifies for namespace distribution, allowing others to receive and use it
- A freshly installed, unvalidated abstraction starts at Isolated tier and must earn its way to User or Namespace tier through demonstrated reliability

---

GUARANTEED CRIME-FREE BUSINESS SERVICES

The Core Idea

The Abstract GT architecture enables a fundamentally new class of service provider: one that can guarantee — not promise, but guarantee — that its service is crime-free. This is structural, not policy-based.

Why "Guaranteed"

In conventional systems, crime prevention relies on policy layers (filters, gates, rules) running on a privileged substrate that can itself be compromised:

- A content filter can be bypassed if the filter code is exploited
- An age gate can be defeated by VPN or cookie manipulation
- A jurisdiction check can be circumvented by spoofed origin headers

The Church Machine achieves guarantees through structural capability scoping:

> A CTMM provisioned without a GT for a service cannot access it, regardless of what code runs on the machine or what exploits are discovered.

There is no filter to bypass, no gate to trick, no policy to manipulate. The capability does not exist.

Scoping by Profession

Each professional domain is a distinct GT set:

```
Medical Professional CTMM holds:
  - GT for medical database (namespace reference)
  - GT for clinical reference service (tunnel channel)
  - GT for regulated comms (tunnel channel)
  - NO GT for financial services
  - NO GT for adult content
  - NO GT for gambling services
```

A medical professional's software simply has no path to financial trading platforms, legal databases, or gambling services. Not because of a policy, but because the GT doesn't exist.

Scoping by Language

Language routing is a GT boundary:

```
French-language service abstraction holds:
  - GT for French service endpoint (tunnel channel 0xFF000001)
  - GT for French data repository (namespace)
  - NO GT for English service endpoint
```

An abstraction provisioned for French-language service cannot reach English-language equivalents. The capability is absent.

Scoping by Nationality and Jurisdiction

Data residency and legal boundaries are GT boundaries:

```
EU CTMM holds:
  - GT for EU data center (tunnel channel)
  - GT for GDPR-compliant services (tunnel channel)
  - NO GT for US-only services
  - NO GT for non-GDPR endpoints
```

An EU-provisioned CTMM cannot reach non-GDPR-compliant services because it was never given the GT for them. The IDE enforces this at provisioning time, and hardware enforces it at runtime.

Scoping by Age Group

Child-safe operation is structural:

```
Child CTMM (age 8) holds:
```

```
  - GT for educational content services
  - GT for parental-approved websites
  - GT for homework-helper endpoints
  - NO GT for adult content services
  - NO GT for social media (unmoderated)
  - NO GT for financial/gambling services
  - NO GT for unvetted remote services
```

A child's CTMM has no capability to reach adult services. Injected JavaScript cannot conjure a GT. Malicious links cannot create tunnels. VPN tools cannot bypass this — the capability is missing at the hardware level.

This is qualitatively different from any filter-based child protection system. A filter examines content after the child already has the capability to fetch it. The Church Machine prevents the capability from existing in the first place.

SECURITY PROPERTIES AND HARDENING ROADMAP

Phase 1 Hardening (Immediate, Non-Silicon)

To address identified security gaps, the following Phase 1 mitigations are non-blocking and ready for implementation:

1. Home Base Threshold Signing — MTBF threshold payloads are signed by IDE public key; CTMM validates signatures before installing
2. Immutable Threshold Ledger — Every threshold change logged to append-only NVM; boot verifies current threshold matches latest ledger entry
3. MTBF Snapshot Validation — Counters are captured in NVM on first abstraction run; reset attempts are detected and flagged as Suspicious
4. CRC Failure Watchdog — CRC validation failures per NS entry tracked; Poisoned entries FAULT on all access after 3 failures
5. Rate-Limiting on mLoad/SWITCH — Max 3 contention retries per instruction before TRAP, preventing timing side-channel attacks
6. Backup Address Validation — Hardware ensures Word 2 ≠ Word 1, Word 3 distinct from both, with recently-tried queue preventing routing loops
7. Chain-of-Custody Logging — Every mSave propagation logged with sender, recipient, timestamp for forensics
8. Wraparound Safety — Minimum GC interval prevents rapid gt_seq wraparound exploitation

Phase 2 Hardening (Full Architecture)

Comprehensive fixes requiring hardware/NS layout changes:

1. Dual-Root Home Base Trust — Two independent Home Base GTs, both must sign

policy (2-of-2 threshold)

2. Tamper-Proof MTBF Counters — Hardware-owned, non-writable except by CALL/RETURN microcode
3. Leased GT Validity — Time-limited GT validity (TTL) for automatic revocation of propagated copies
4. Cryptographic Integrity — HMAC-SHA256 or dual CRC-32 replacing 16-bit CRC
5. Per-Recipient Revocation — Bloom filter in NS entry enabling selective revocation of propagated GTs

---

DETAILED CLAIMS (NOVEL)

Claim 1: Abstract Golden Token with Hardware-Routed Sentinel Address (Independent)

A capability register architecture wherein a Golden Token of type Abstract (gt_type = 11_2) contains a word1_location field that is a 32-bit sentinel address outside the real RAM range, and wherein hardware matches this address directly against an Abstract Address Space table rather than performing a namespace lookup, such that the address alone constitutes the token's identity and unforgeability, without any namespace slot allocation, CRC validation, or lump header dereference.

Claim 2: Home Base Tunnel as Single Network Gateway (Independent)

An I/O virtualization mechanism wherein a single Abstract GT (Home Base tunnel at address 0xFF000000) is the sole outbound network interface, with all network access from all abstractions routed through this single hardware-managed tunnel, and wherein the Home Base GT can be provisioned with optional programmer-defined backup IDE addresses (Word 2 and Word 3) that are tried in sequence if the primary address is unreachable, with all three addresses set atomically at boot and immutable thereafter.

Claim 3: Local Peripheral Autonomy Without IDE Assistance (Independent)

A hardware bootstrap mechanism wherein attached peripherals (UART, GPIO, Timer, Display, Storage) are identified and Abstract GTs are provisioned entirely during local hardware boot, before any user code runs and before any IDE connection is established, such that the CTMM maintains full security control over local I/O without any remote authority, enabling air-gapped, offline operation where the CTMM is the sole authority for its own hardware security policy.

Claim 4: MTBF Qualification as Hardware-Enforced Downloadability Gate (Independent)

A hardware-tracked reliability mechanism wherein every Secure Abstraction carries invocation_count and failure_count counters, an MTBF score is computed as invocation_count / (failure_count + 1), and the S (Save) permission on the abstraction's GT is hardware-locked based on the MTBF tier (Isolated, User-regulated, Namespace-regulated), such that unreliable abstractions cannot propagate beyond their provisioning context, and the IDE remotely updates tier thresholds via signed policy payloads delivered through the Home Base tunnel.

Claim 5: Structural Capability Scoping for Crime-Free Services (Novel)

A service provisioning architecture wherein a CTMM provisioned for a specific profession, language, nationality, or age group receives only Abstract GTs for services within that scope, such that any code running on the CTMM cannot access services outside the scope because the capability token does not exist, providing a structural guarantee (not a policy-based filter) that access is impossible regardless of exploits, VPN tunnels, or code injection — because the hardware rejects any operation without a valid GT.

Claim 6: Two-Tier Backup IDE Fallback with Atomic Provisioning (Dependent)

An extension of Claim 2 wherein the Home Base GT includes word2_backup1 and word3_backup2 fields, each containing a 32-bit Abstract Address within the tunnel range, and hardware implements sequential failover: try primary → try Word 2 if reachable → try Word 3 if reachable → TRAP: TUNNEL_UNAVAILABLE, with loop detection ensuring no two addresses are identical and no recently-tried address is re-attempted within the same connection attempt.

Claim 7: MTBF Telemetry and Distributed Reputation (Dependent)

An extension of Claim 4 wherein MTBF counters are sent to the IDE via Home Base W permission, signed with HMAC-SHA256, and the IDE maintains a permanent reputation record per abstraction per CTMM, such that an abstraction's reliability history is shared across the fleet: low MTBF on one CTMM lowers the threshold for all CTMMs through policy update, and high MTBF qualifies abstractions for namespace-tier distribution.

Claim 8: Secure Individuality via Unique GT Sets (Dependent)

A trust isolation mechanism wherein each abstraction's identity is defined as the unique set of GTs provisioned in its c-list, such that no two abstractions share ambient authority, an attacker who compromises one abstraction gains only its GTs and cannot escalate to another abstraction's resources without holding that abstraction's GTs, and the "privileged superuser" attack window is eliminated entirely — there is no privileged layer that holds authority on behalf of others.

DENIAL OF SERVICE PREVENTION AND CONTROL MECHANISMS (NOVEL)

The DoS Problem in Conventional Systems

Denial of Service attacks exploit several fundamental weaknesses in contemporary architectures:

1. Unbounded Resource Access — Any code can request arbitrary resources (file I/O, network bandwidth, memory allocation) without proving it holds the right to them.
2. Privilege Escalation via DoS — A low-privilege process can exhaust system resources, forcing the OS to deny service to higher-privilege processes. The OS itself becomes the bottleneck.
3. No Per-Resource Rate Limiting — All access to a resource goes through a shared bottleneck (e.g., the network stack, the filesystem). An attacker can saturate the bottleneck for all users.
4. Ambient Authority Sharing — Multiple processes share TCP/IP stack, file descriptor table, memory allocator. An attack on any shared resource affects all processes.
5. Cascade Failures — When one service is DoS'd, dependent services starve. The attack propagates through the system.

How CTMM Prevents DoS

The Church Machine architecture prevents or drastically limits DoS attacks through structural mechanisms:

Mechanism 1: Capability-Gated Resource Access

Only code that holds an Abstract GT for a resource can access it. To launch a DoS attack on a resource, the attacker must first hold the capability.

```
Example: UART Service DoS Prevention
- UART is provisioned as Abstract GT 0xFE000001 (local peripheral)
- Only abstractions with GT for 0xFE000001 can access UART
- Attacker code without the GT cannot request any UART operations
- UART DoS is impossible without holding the capability
```

Impact: An attacker in one abstraction cannot DoS a resource that abstraction was not given access to.

Mechanism 2: Per-Abstraction Isolation

Each abstraction has its own c-list with its own GT set. Resources are not ambient or shared by default.

```
Two abstractions running concurrently:
  Abstraction A: holds GT for Service X only
```

```
  Abstraction B: holds GT for Service Y only

If A is DoS'd by local attacker:
  - B continues running normally (no shared resources exhausted)
  - Service Y remains responsive
  - Only Service X is affected

The attack is contained to the attacker's own abstraction.
```

Impact: One compromised abstraction cannot cascade-DoS other abstractions.

Mechanism 3: Hardware-Enforced mLoad Retry Limits

All namespace access routes through mLoad, which has a maximum of 3 contention retries per instruction before raising TRAP: MAX_RETRIES_EXCEEDED.

```
mLoad retry sequence:
  Attempt 1: LOAD instruction retries (contention)
  Attempt 2: LOAD instruction retries again (contention)
  Attempt 3: LOAD instruction retries once more (contention)
  Attempt 4: TRAP: MAX_RETRIES_EXCEEDED (IRQ handler decides next action)
```

An attacker cannot hammer the namespace gate with unlimited retries. The attacker must explicitly wait (exponential backoff) before retrying.

Impact: Rate-limited access prevents namespace gate exhaustion.

Mechanism 4: Hardware-Enforced SWITCH Rate Limits

SWITCH instruction (privilege gate for CR13/CR15) has the same 3-retry limit as mLoad.

```
SWITCH CR_passkey, CR15
  Attempt 1: contention, retry
  Attempt 2: contention, retry
  Attempt 3: contention, retry
  Attempt 4: TRAP: MAX_RETRIES_EXCEEDED
```

An attacker cannot force rapid privilege switching that could disrupt scheduling or timing.

Impact: Prevents privilege escalation DoS attacks.

Mechanism 5: MTBF Qualification Prevents DoS Propagation

An abstraction that crashes or fails frequently (high failure_count) is marked Isolated by hardware and cannot be propagated to other CTMMs.

```
Malicious abstraction crashes 50% of the time:
  MTBF score = 100 / 50 = 2.0
  Threshold for User-tier: 0.99
  Score 2.0 > 0.99 → Passes user-tier threshold initially

After 1000 invocations, with 50% failure rate:
  MTBF score = 1000 / 500 = 2.0
  Score does not improve
  IDE updates threshold: user_tier = 0.999 (tighten)
  Score 2.0 > 0.999 → Still passes, but barely

After 10,000 invocations:
  If failure pattern holds: MTBF = 10000 / 5000 = 2.0
```

```
  IDE updates: user_tier = 0.9999
  Score 2.0 > 0.9999 → FAILS threshold
  S bit locks to 0: no further propagation
```

A DoS-prone abstraction cannot escape its origin and infect other CTMMs.

Impact: DoS attacks are quarantined to the originating CTMM.

Mechanism 6: Local Peripheral Rate Limiting

Hardware-controlled access to local peripherals (UART, GPIO, Timer, Display) can enforce per-abstraction bandwidth limits or operation counts.

```
UART rate limiting example:
- Each abstraction holds a GT for the UART with R/W permissions
- Hardware enforces per-abstraction quota: max 1000 bytes per second
- If abstraction exceeds quota, further UART operations are deferred
- Other abstractions continue UART access at normal rate
```

Impact: Local I/O DoS attacks are rate-limited per abstraction.

Mechanism 7: Home Base Tunnel Flow Control

The Home Base tunnel (single network gateway) implements flow control at the hardware level:

```
Home Base tunnel bandwidth management:
- Per-CTMM outbound quota: X Mbps max
- Per-abstraction outbound quota: Y Kbps max
- Per-connection timeout: Z seconds
- If quota exceeded, further sends are deferred (backpressured)
```

An attacking abstraction cannot starve other abstractions of network bandwidth because each abstraction has its own quota.

Impact: Network DoS attacks are rate-limited per abstraction.

Mechanism 8: GC Interval Safety Limits

Garbage collection has a minimum interval (e.g., 1 second) between wraparound events. If GC tries to wrap the gt_seq counter too frequently, a TRAP fires.

```
gt_seq wraparound interval limit:
  Last wraparound at time T
  Next wraparound attempt at time T + 500ms
  500ms < 1 second minimum
  TRAP: GC_WRAPAROUND_TOO_FAST
```

An attacker cannot force rapid GC cycles to accelerate wraparound and confuse stale GTs with fresh ones.

Impact: GC-based DoS attacks are prevented.

Mechanism 9: Abstract Address Validation

Every attempt to use an Abstract GT requires hardware validation of the address. Invalid

addresses are TRAP'd:

```
Hardware validation on Abstract GT operation:
  word1_location = 0x12345678  (not in reserved range)
  Check: is 0x12345678 in Abstract Address Space?
  No → TRAP: INVALID_ABSTRACT_ADDRESS
```

An attacker cannot cause DoS by constructing bogus Abstract Addresses.

Impact: Malformed operations fail fast without consuming resources.

Claim 9: Capability-Gated DoS Prevention (Independent)

A denial-of-service prevention mechanism wherein access to any resource is controlled exclusively through unforgeable capability tokens, such that an attacker without the capability token for a resource cannot even request operations on that resource, and thus cannot launch a DoS attack on that resource, with the constraint being structural and enforced at the hardware level rather than through rate-limiting or filtering.

Claim 10: Per-Abstraction Resource Isolation (Independent)

A resource isolation mechanism wherein each Secure Abstraction has its own c-list with its own GT set, and resources (I/O, network, memory allocators, queues) are not shared by default, such that an attacking abstraction that exhausts its own resources affects only itself and does not cascade to other abstractions, enabling DoS containment at the abstraction boundary.

Claim 11: Hardware-Enforced Retry Limits on Critical Paths (Independent)

A rate-limiting mechanism wherein critical paths (mLoad for namespace access, SWITCH for privilege gate, GC for garbage collection) enforce a maximum of three contention retries before raising a TRAP that returns control to the IRQ handler, and wherein the hardware counter resets only on successful completion or explicit FAULT, such that an attacker cannot hammer critical paths without explicit wait periods (exponential backoff), preventing saturation DoS attacks on the Trusted Security Base.

Claim 12: MTBF Qualification as DoS Containment (Dependent)

An extension of Claim 4 wherein an abstraction that fails frequently (high failure_count) is automatically downgraded to Isolated tier and cannot be propagated to other CTMMs, such that DoS-prone or malicious abstractions cannot spread across the fleet through normal capability propagation, and fleet-wide policy updates further tighten MTBF thresholds to quarantine unreliable abstractions.

Claim 13: Per-Abstraction Bandwidth Quotas on Tunnel Access (Dependent)

An extension of Claim 2 wherein the Home Base tunnel implements per-abstraction outbound bandwidth quotas (measured in bytes per second or operations per second), and wherein abstractions that exceed their quota are backpressured rather than dropped, such that one attacking abstraction cannot saturate the shared tunnel and starve other abstractions of network access.

Claim 14: Local Peripheral Access Rate Limiting (Dependent)

An extension of Claim 3 wherein access to local peripherals (UART, GPIO, Timer, Display) is rate-limited per abstraction, either through hardware quotas or token-bucket mechanisms, such that an abstraction cannot monopolize peripheral bandwidth and DoS other abstractions' local I/O operations.

CONCLUSION

The Abstract Golden Token I/O and Network Addressing architecture represents a fundamental shift in how computer systems integrate I/O, network access, and service provisioning. By making these capabilities unforgeable, hardware-routed, and locally autonomous (for peripherals) or IDE-provisioned-at-boot (for network), the architecture enables:

1. Structural safety: Services can be scoped at the capability level, guaranteeing that certain code cannot access certain services because the token doesn't exist.
2. Offline autonomy: Local peripherals work without any IDE connection, making CTMMs suitable for air-gapped, autonomous, and edge-deployed scenarios.
3. Trusted fallback: Programmers can configure backup IDE addresses, enabling local-first or team-first development without compromising CTMM security.
4. Distributed reputation: MTBF qualification creates a mesh-style trust model where reliability is tracked and propagated, replacing centralized app stores.
5. Crime-free guarantees: For the first time, a computer system can structurally guarantee that a user's device cannot access prohibited services — not through filters or policies, but through the absence of capability tokens.

This invention builds on the CTMM foundation while opening entirely new application domains: edge computing, offline-first systems, autonomous agents, and the first genuinely crime-free platforms.

ADDENDUM C: LAMBDA RECURSION AND

SELF-INVOCATION (Filed April 2026)

Church-Turing Meta-Machine: O(1) Lambda Recursion Through Self-Invocation via CR6, Idempotent LAMBDA Re-Entry, and Multi-Paradigm Natural Language Compilation

---

Inventor: Kenneth James Hamer-Hodges

Filing Date: April 2026

Parent Application: Church-Turing Meta-Machine: Hardware-Enforced Lambda Calculus with the LAMBDA Instruction, NULL Capability Type, and Atomic Abstraction Architecture (Filed February 12, 2026)

Related Applications:

- Pure Church Lambda Processor: Architectural Exclusion of Turing-Domain Instructions as a Security Enforcement Mechanism (Filed February 2026)
- Church-Turing Meta-Machine: Dual-Gate Trusted Security Base with Hardware-Enforced Lambda Calculus, Deterministic Garbage Collection, and Architectural Vulnerability Elimination (Filed February 2026)
- Language-Independent Capability-Secured Instruction Set Architecture with Multi-Language Compiler (Filed March 2026)
- Abstract Golden Token I/O and Network Addressing Architecture (Filed March 2026)

Classification: Computer Architecture; Hardware Security; Capability-Based Computing; Lambda Calculus Processor; Recursion Optimization; Natural Language Compilation; Context-Switch Optimization

---

TITLE OF THE INVENTION

O(1) Lambda Recursion Through Idempotent Self-Invocation via Capability List Register in a Capability-Based Processor with Natural Language Compilation

---

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is a continuation-in-part of the CTMM patent applications filed February–March 2026, which disclose: the Golden Token (GT) capability architecture with domain purity enforcement; the LAMBDA instruction for lightweight in-scope code application with machine-status fast path; self-describing stack frames with 1-bit

CALL/LAMBDA tag; non-nestable LAMBDA with CALL-mediated nesting; the CALL lump split creating CR14 (code, X-only) and CR6 (c-list, L-only); and the CLOOMC++ multi-language compiler with JavaScript and Haskell front-ends.

The present application extends those disclosures with five innovations:

1. Self-invocation via CR6 — A recursion mechanism where CALL CR6 and LAMBDA CR6 re-enter the current method using the capability list register that the CALL lump split already established, providing zero-cost self-reference without additional GT allocation or namespace lookup.

2. Idempotent LAMBDA re-entry — A refinement of the parent application's non-nestable LAMBDA rule, wherein LAMBDA CR6 is permitted to re-execute while lambda_active is already set, because the return address is invariant (the same value is overwritten with the same value). No hardware counter is needed — the software's own recursion argument (e.g., n counting down to 0) drives the recursion, and the existing 1-bit lambda_active flag plus lambda_pc register provide O(1) exit: the base-case RETURN clears the flag and jumps to the instruction after LAMBDA CR6, where a second RETURN pops the CALL frame. Two RETURNs total, regardless of depth.

3. Three architectural loop styles — While loops (BRANCH), Recursive Repeat (CALL CR6), and Lambda Recursion (LAMBDA CR6) as distinct hardware mechanisms offering different security, performance, and resource tradeoff profiles, all compilable from the same source language.

4. Natural language compilation — An English front-end for the CLOOMC++ compiler that accepts plain English sentences ("Add a method called Sum that takes n", "While n is greater than 0", "Repeat with n minus 1") and compiles them to capability-secured Church Machine instructions.

5. Pet-name capability references — Named capability tokens (Pi, E, Phi, Zero, One) compiled as Abstract GT operations, enabling mathematical constant references in compiled code through the capability system.

FIELD OF THE INVENTION

The present invention relates to recursion optimization in a capability-based processor, wherein the architectural properties of the LAMBDA instruction — specifically, the invariance of the return address during self-invocation — enable idempotent re-entry without additional hardware state. The software's own recursion argument drives the countdown; the hardware's existing lambda_active flag and lambda_pc register handle the O(1) exit path. No hardware counter, no stack frames, and no additional registers are required. The invention further relates to a self-invocation mechanism using the capability list register (CR6) and to natural language compilation targeting the capability-secured instruction set.

BACKGROUND

The Recursion Cost Problem

All known processor architectures implement recursion through stack frames. Each recursive call pushes a return address (and typically saved registers) onto the stack. For N recursive calls, the architecture consumes O(N) stack space, requires O(N) time to unwind on return, and requires O(N) time to save/restore on context switch. This cost applies regardless of whether the recursion is deep or shallow, and regardless of whether the return addresses are all the same or all different.

The Parent Architecture's Recursion Model

The parent CTMM application discloses two code invocation mechanisms:

- CALL (E permission): Crosses a protection domain boundary. Pushes a 2-word stack frame (E-GT + machine word with NIA). Performs namespace gate swap, creating new CR14 (code) and CR6 (c-list). Full mLoad validation on return. Heavyweight but secure.
- LAMBDA (X permission): Stays within the current protection domain. In the common case (no intervening CALL), uses a machine-status register pair (LAMBDA_PC and LAMBDA_active flag) with zero stack access. Lightweight but non-nestable — a second LAMBDA while LAMBDA_active is set causes a FAULT.

The parent application describes LAMBDA as non-nestable, with controlled nesting possible only through CALL mediation. Each nesting level requires a full CALL/RETURN pair. This is correct and secure, but imposes O(N) overhead for recursive computations.

The Discovery: Return Address Invariance

The present invention recognizes that when a method invokes itself via CR6 using LAMBDA, every recursive call returns to the same address — the instruction immediately after the LAMBDA CR6 instruction. This invariance means that re-executing LAMBDA CR6 while lambda_active is already set is idempotent: it overwrites lambda_pc with the same value it already holds. The non-nestable FAULT is unnecessary for self-invocation because no new return context is created — the same return address is being written again.

The Key Insight: The Software Already Counts

The recursion depth is not hidden from the software — it is the software. The method's own argument (e.g., n in Sum(n)) counts down to zero. The software determines when to stop (the base case). The hardware does not need to independently track depth because the software already does. The hardware only needs to know one thing: "am I in a LAMBDA body?" — which is the existing 1-bit lambda_active flag.

This insight eliminates the need for any hardware counter:

1. The software counter (n) drives the recursion
2. The base case (n == 0) triggers RETURN
3. RETURN sees lambda_active = 1, restores PC from lambda_pc, clears the flag
4. The restored PC points to the instruction after LAMBDA CR6, which is RETURN
5. This second RETURN sees lambda_active = 0, pops the CALL frame
6. Two RETURNs. Always two. Regardless of recursion depth.

The Self-Invocation Discovery

The parent application discloses the CALL lump split: when CALL processes an E-GT, it creates CR14 (code region, X-only) and CR6 (c-list region, L-only). The present invention recognizes that CR6 — which CALL has already established to point at the current method's c-list — can serve as a self-reference for recursion.

- CALL CR6: Invokes the current method again through the full CALL path (namespace gate swap, 2-word frame, capability re-validation). This is Recursive Repeat — secure, predictable, heavyweight.
- LAMBDA CR6: Invokes the current method again through the lightweight LAMBDA path (no namespace swap, machine-status register, X permission check only). This is Lambda Recursion — the lightest possible self-invocation.

In both cases, CR6 was already set up by the initial CALL that entered the method. No additional GT allocation, no namespace lookup, no c-list modification is required. The self-reference is a free consequence of the architecture.

DETAILED DESCRIPTION

1. Self-Invocation via CR6

1.1 The Mechanism

When a method is entered via CALL, the CALL lump split creates:

- CR14: Code region (X-only), pointing to the method's instruction space
- CR6: C-list region (L-only), pointing to the method's capability list

CR6 holds an Inform GT with L (Load) permission pointing to the method's c-list base. The c-list IS the method's entry point for self-reference — because CALL CR6 will re-enter the same lump, re-split it, and re-create the same CR14 and CR6.

CALL CR6 (Recursive Repeat):

```
Step 1: Read CR6 — Inform GT with L permission, pointing to current method's c-list
Step 2: CALL processes CR6 as an E-GT (the method's lump has E permission)
Step 3: CALL performs lump split: creates new CR14 and CR6 (identical to current)
Step 4: Push 2-word frame (E-GT + machine word with NIA)
Step 5: Branch to method entry (offset 0 in new CR14)
```

```
Step 6: Method body executes with fresh CR14/CR6 (same values, new frame)
Step 7: RETURN pops frame, restores caller's CR14/CR6, validates via mLoad
```

LAMBDA CR6 (Lambda Recursion — Idempotent Re-Entry):

```
Step 1: Read CR6 — Inform GT with X permission (code body in same domain)
Step 2: LAMBDA verifies X permission on CR6
Step 3: If lambda_active == 0:
          Save return address (PC+4) to lambda_pc
          Set lambda_active = 1
        If lambda_active == 1:
          Overwrite lambda_pc with PC+4 (same value — idempotent)
Step 4: Branch to code entry point referenced by CR6
Step 5: Method body executes (same CR14, same CR6 — no namespace swap)
Step 6: On base case RETURN:
          lambda_active = 1 → PC ← lambda_pc, clear lambda_active
          Falls to instruction after LAMBDA CR6
Step 7: Second RETURN:
          lambda_active = 0 → real RETURN, pop CALL frame
```

1.2 The Idempotent Re-Entry Rule

The parent application's non-nestable rule states: if lambda_active is set and a LAMBDA instruction executes, the hardware generates a FAULT. The present invention refines this rule with a self-invocation exception:

Refined Rule: If lambda_active is set and LAMBDA CR6 executes (self-invocation), the hardware permits re-entry because the return address is invariant — lambda_pc is overwritten with the same value it already holds. This is not nesting (no new return context is created); it is re-entry to the same body with updated data register arguments.

The FAULT is preserved for LAMBDA to a different target (different CR, different return address) while lambda_active is set. The distinction is:

- LAMBDA CR6 while active → permit (idempotent, same return address)
- LAMBDA CRn (n ≠ 6) while active → FAULT (different return address, true nesting)

This requires no new hardware — it is a refinement of the existing FAULT condition logic, gating on whether the target CR is CR6.

1.3 Traced Execution: LambdaSum(3, 0)

```
CALL enters LambdaSum → push CALL frame, lump split creates CR14 + CR6
  lambda_active = 0

Level 0: DR_n=3, DR_total=0
  n≠0, skip base case
  total = 0+3 = 3, n = 3-1 = 2
  LAMBDA CR6: lambda_active was 0 → set lambda_active=1, lambda_pc=addr_after_lambda
  Branch to method entry

Level 1: DR_n=2, DR_total=3
  n≠0, skip base case
  total = 3+2 = 5, n = 2-1 = 1
  LAMBDA CR6: lambda_active already 1 → overwrite lambda_pc with same value (idempotent)
  Branch to method entry

Level 2: DR_n=1, DR_total=5
  n≠0, skip base case
```

```
  total = 5+1 = 6, n = 1-1 = 0
  LAMBDA CR6: lambda_active already 1 → idempotent
  Branch to method entry

Level 3: DR_n=0, DR_total=6
  n==0! Base case.
  RETURN #1: lambda_active=1 → PC ← lambda_pc, clear lambda_active
    DR_total=6 (correct answer!)
    Now at addr_after_lambda

  RETURN #2: lambda_active=0 → real RETURN
    Pop CALL frame from initial entry
    Back to caller with DR_total=6 ✓

Total: 2 RETURNs for depth 3. Same 2 RETURNs for depth 3,000,000.
No counter. No stack frames. No unwinding.
```

1.4 Why No Hardware Counter Is Needed

The initial design proposed a 16-bit hardware counter (lambda_depth_reg) to track recursion depth. Analysis reveals this counter is unnecessary:

1. The software already counts: The method's own argument (n) is the recursion counter. It counts down to zero. The hardware does not need to independently replicate this count.

2. The counter would be discarded: When the base-case RETURN fires, the proposed counter would be zeroed and thrown away. It served no purpose during execution — the software determined when to stop.

3. The existing flag is sufficient: The 1-bit lambda_active flag already tells RETURN whether to take the LAMBDA fast path. No additional depth information is needed because the fast path behavior is the same regardless of depth: restore PC from lambda_pc, clear flag, done.

4. Two RETURNs is already O(1): The base-case RETURN clears the flag (O(1)), the follow-on RETURN pops the CALL frame (O(1)). Total: O(1) regardless of depth. A counter cannot improve on this.

5. Context switch is already O(1): CHANGE saves lambda_active (1 bit) and lambda_pc (already packed in the NIA word) — two fixed-size values regardless of recursion depth. No counter needed.

The simpler design — just relaxing the FAULT rule for CR6 self-invocation — achieves the same O(1) trifecta with zero additional hardware.

1.5 Compiler Primitives

The CLOOMC++ compiler provides two primitives for self-invocation:

- recall(args): Emits LOAD instructions for updated arguments followed by CALL CR6. English syntax: "Repeat with x, y", "Recurse with n minus 1", "Call self with a, b", "Call again with n, total".

- relambda(args): Emits LOAD instructions for updated arguments followed by LAMBDA CR6. English syntax: "Apply lambda with x, y", "Lambda repeat with n minus 1", "Lambda recurse with a, b", "Lambda self with count".

1.6 Security Properties — CALL CR6 Adds No Security Over LAMBDA CR6

A critical architectural observation: CALL CR6 is no more secure than LAMBDA CR6 for self-invocation. Both re-enter the same method, in the same protection domain, with the same c-list, accessing the same capabilities.

- CALL CR6 performs full namespace gate re-validation on each recursive call — but the gate re-validates the same GT, re-splits the same lump, creates the same CR14 and CR6, and enters the same code body. The re-validation is redundant work that produces an identical result every time. No new capabilities are introduced. No new domain is entered. The "security" of per-iteration re-validation is illusory when the target never changes.
- LAMBDA CR6 verifies X permission before branching — once. Since every re-entry targets the same code body with the same capabilities in the same domain, the X check on the first entry is sufficient. Subsequent idempotent re-entries execute the same verified code.
- No GT forgery in either case: CR6 was established by the original CALL lump split. Software cannot modify CR6's GT — only CALL and mLoad can write to capability registers. Both CALL CR6 and LAMBDA CR6 use a GT that was already validated at method entry.
- Idempotent re-entry is safe: Each re-entry executes the same code body with the same capabilities. Only data registers change (the arguments). The capability register file is untouched by LAMBDA execution — this is the Golden Rule strengthened, as disclosed in the parent application.

Conclusion: CALL CR6 recursion is strictly inferior to LAMBDA CR6 recursion — identical security, but O(N) stack cost, O(N) context-switch cost, and O(N) unwind cost versus O(1) for all three with LAMBDA CR6. CALL CR6 is never the preferred choice for self-invocation. It exists in the architecture because CALL can target any CR (including CR6), but when the intent is recursion, LAMBDA CR6 dominates on every axis. The only reason to describe CALL CR6 recursion is to demonstrate the superiority of LAMBDA CR6 by contrast.

2. The O(1) Trifecta — Without a Counter

2.1 O(1) Entry

Each LAMBDA CR6 re-entry requires:

- X permission check on CR6 (combinational — no memory access)
- Overwrite lambda_pc with PC+4 (same value — idempotent write to existing register)
- Branch to method entry

No stack frame push. No counter increment. No memory access. One clock cycle beyond the normal LAMBDA path.

2.2 O(1) Context Switch

On CHANGE (thread context switch), the hardware saves:

- lambda_active (1 bit) — already packed in the NIA word's indicator bits
- lambda_pc (32 bits) — already saved as part of machine status

These are two fixed-size values regardless of recursion depth. Whether the method has recursed 1 time or 1,000,000 times, CHANGE saves the same two values. Compare with CALL CR6 recursive repeat, where each of N CALL frames must be individually saved.

2.3 O(1) Exit

The base case triggers two RETURNs:

RETURN #1 (from base case):

- lambda_active = 1 → restore PC from lambda_pc, clear lambda_active
- This is the existing LAMBDA fast path from the parent application
- One cycle, no stack access

RETURN #2 (from after-LAMBDA instruction):

- lambda_active = 0 → real RETURN, pop CALL frame
- This is the normal CALL RETURN path
- Standard frame pop

Total: exactly 2 RETURN instructions regardless of recursion depth.

For recursion depth N, this eliminates N-1 RETURN instructions compared to an architecture that unwinds iteratively. For N = 1,000,000, this saves 999,999 RETURN cycles.

2.4 The Counter as Optional Enhancement

While the core mechanism requires no hardware counter, an optional counter register (lambda_depth_reg, 16 bits) could serve diagnostic purposes:

- Performance monitoring: Query recursion depth without walking the stack
- Safety limit: Generate a FAULT on counter overflow (e.g., depth > 65535) to catch runaway recursion before stack exhaustion on the CALL side
- Debugging: Hardware watchpoint on counter value

These are enhancement features, not requirements for correctness or O(1) performance. The base mechanism — idempotent LAMBDA re-entry with the existing flag and PC register — is complete without the counter.

3. Three Architectural Loop Styles

3.1 The Discovery

The combination of BRANCH, CALL CR6, and LAMBDA CR6 provides three distinct loop mechanisms with different security, performance, and resource profiles — all compiling from the same source code, all producing correct results, but with fundamentally different hardware behavior.

3.2 Comparison

Property	While (BRANCH)	CALL CR6 (shown for contrast)	LAMBDA CR6 (optimal)
Opcode	MCMP + BRANCH	CALL	LAMBDA
Loop mechanism	Compare-and-branch	Self-invocation with full frame	Idempotent self-invocation
Stack per iteration	0	2 words (SZ=1)	0 (no stack, no counter)
Namespace gate swap	No	Yes (redundant — same GT every time)	No
Branch prediction	Required (misprediction risk)	Not required (target is CR6, known)	Not required (target is CR6, known)
Speculative execution	Yes (Spectre/Meltdown risk)	No	No
Context switch cost	O(1)	O(N) (save N frames)	O(1) (save flag + PC)
Unwind cost	O(1)	O(N) (pop N frames)	O(1) (2 RETURNs always)
Pipeline stall risk	Yes (misprediction on final iteration)	No	No
Security vs LAMBDA	Requires branch analysis	No additional security (same method, same domain, same entry)	Optimal security (same entry)
Instructions per iteration	4+ (MCMP, BRANCH, body, BRANCH back)	2+ (args, CALL)	2+ (args, LAMBDA)
Additional hardware	Branch predictor	Stack memory	None (existing flag + PC)
Verdict	Vulnerable (Spectre/Meltdown)	Strictly inferior to LAMBDA CR6	Optimal for recursion

3.3 Architectural Significance — LAMBDA CR6 Dominates

The three loop styles exist in the architecture, but they are not equal alternatives — they form a hierarchy where LAMBDA CR6 is the clear winner for recursive self-invocation:

- While (BRANCH): Familiar to imperative programmers. Most compact loop code. But inherits all of conventional computing's branch prediction problems: pipeline stalls on misprediction, speculative execution vulnerabilities (Spectre, Meltdown), non-deterministic timing. These are the exact vulnerabilities the Church Machine is designed to eliminate.
- CALL CR6 (Recursive Repeat): Performs full namespace gate re-validation on every iteration — but this re-validation is redundant because CR6 always points to the same method. The gate re-checks the same GT, re-splits the same lump, and produces the same CR14 and CR6 every time. CALL CR6 adds no security over LAMBDA CR6 while imposing O(N) stack, O(N) context switch, and O(N) unwind costs. CALL CR6 is never the preferred choice for recursion. It is included in this comparison solely to demonstrate, by contrast, the power of LAMBDA CR6.
- LAMBDA CR6 (Lambda Recursion): O(1) everything — entry, context switch, exit. No branch prediction, no speculative execution, no namespace swap. No additional hardware beyond the existing lambda_active flag and lambda_pc register. Same security as CALL CR6 (same method, same domain, same capabilities) with none of the overhead. The optimal recursion primitive.

The architectural lesson: CALL is the correct instruction for cross-domain invocation (entering a different abstraction with a different c-list). When the target is the same method (CR6), CALL's heavyweight machinery provides no benefit — it is LAMBDA's domain. The CALL lump split creates CR6 as a self-reference; LAMBDA CR6 is how that self-reference should be used.

4. Natural Language Compilation (English Front-End)

4.1 The Extension

The parent CLOOMC++ patent application discloses JavaScript (imperative) and Haskell (functional) front-ends compiling to the 20-instruction Church Machine ISA. The present invention adds an English front-end — the third paradigm — that accepts plain English sentences and compiles them to the same capability-secured instruction set.

4.2 English Syntax

The English front-end parses structured English sentences with the following mappings:

English Source	Church Machine Instruction(s)
Add a method called X that takes n	Method prologue (register allocation for parameters)
Set total to 0	IADD DRx, DR0, #0 (DR0 = hardwired zero)
Set total to total plus n	IADD DRx, DRy, DRz
Set n to n minus 1	ISUB DRx, DRy, #1
While n is greater than 0	MCMP DRx, #0 + BRANCH.LE (skip to End while)
End while	BRANCH (jump back to While)
If n is equal to 0	MCMP DRx, #0 + BRANCH.NE (skip to End if)
End if	(branch target)
Return total	Move result to DR1 + RETURN AL
Repeat with n, total	LOAD args + CALL CR6 (recall)
Apply lambda with n, total	LOAD args + LAMBDA CR6 (relambda)

4.3 Condition Phrases

The English compiler recognizes six comparison phrases, each mapping to an MCMP condition code:

English Phrase	MCMP Condition
is greater than	GT
is less than	LT
is equal to / equals	EQ
is not equal to	NE
is greater than or equal to	GE
is less than or equal to	LE

4.4 Universal Target Proof

The English front-end produces the same instructions as the JavaScript and Haskell

front-ends for equivalent computations:

- English: Set total to total plus n → IADD DRx, DRy, DRz
- JavaScript: total = total + n → IADD DRx, DRy, DRz
- Haskell: total + n → IADD DRx, DRy, DRz

The hardware cannot distinguish which language produced the code. This extends the universal computation target property to three paradigms: imperative (JavaScript), functional (Haskell), and natural language (English).

4.5 Significance for the Three Loop Styles

The English front-end is the first natural language compiler to directly expose all three architectural loop styles:

- While: While n is greater than 0 ... End while → MCMP + BRANCH
- Recursive Repeat: Repeat with n, total → CALL CR6
- Lambda Recursion: Apply lambda with n, total → LAMBDA CR6

A programmer writing in plain English can choose between conventional looping, secure recursion, and lightweight lambda recursion — without knowing assembly language, register names, or opcode encodings.

5. Pet-Name Capability References

5.1 The Mechanism

The CLOOMC++ compiler recognizes mathematical constant names (Pi, E, Phi, Zero, One) and compiles them as references to Abstract Golden Tokens in the Lambda abstraction's c-list. Each constant is an Abstract GT (gt_type = 11_2) whose word1_location encodes the constant's hardware identity.

5.2 Compilation

```
Source:  Load Pi
Target:  LOAD CR_temp, CR6, #offset_of_Pi_in_clist
         ; CR_temp now holds Abstract GT for Pi
         ; Value encoded in word1_location, immutable, unforgeable
```

5.3 Significance

Pet-name capability references demonstrate that the Abstract GT type — originally designed for I/O addressing — also serves as a mechanism for mathematical constants. The constant's value is architecturally immutable (it is the GT, not a memory location), unforgeable (no instruction can synthesize a GT), and capability-secured (requires L permission on CR6 to access).

REDUCTION TO PRACTICE

Self-Invocation Proof

The CLOOMC++ compiler (simulator/cloomc_compiler.js) implements recall() and relambda() primitives. The following three methods compute the sum 1+2+...+n using all three loop styles:

WhileSum (BRANCH, 12 instructions):

```
Add a method called WhileSum that takes n
Set total to 0
While n is greater than 0
    Set total to total plus n
    Set n to n minus 1
End while
Return total
```

Compiles to: IADD, MCMP, BRANCH.LE, IADD, ISUB, BRANCH, RETURN — 2 branches per iteration.

RecurseSum (CALL CR6, 10 instructions):

```
Add a method called RecurseSum that takes n and total
If n is equal to 0
    Return total
End if
Set total to total plus n
Set n to n minus 1
Repeat with n, total
```

Compiles to: MCMP, BRANCH.NE, RETURN, IADD, ISUB, LOAD, LOAD, CALL CR6 — 1 CALL per iteration, 0 branches for looping.

LambdaSum (LAMBDA CR6, 10 instructions):

```
Add a method called LambdaSum that takes n and total
If n is equal to 0
    Return total
End if
Set total to total plus n
Set n to n minus 1
Apply lambda with n, total
```

Compiles to: MCMP, BRANCH.NE, RETURN, IADD, ISUB, LOAD, LOAD, LAMBDA CR6 — 1 LAMBDA per iteration, 0 branches for looping, zero stack overhead (idempotent re-entry, no frames pushed).

All three methods produce Sum(5) = 15. The compiled code has been verified in the Church Machine simulator.

Idempotent Re-Entry Verification

The traced execution of LambdaSum(3, 0) demonstrates:

- 3 levels of LAMBDA CR6 re-entry with lambda_active already set
- Each re-entry overwrites lambda_pc with the same value (idempotent)
- Base case triggers 2 RETURNs: flag-clear + CALL-frame-pop
- Result: DR_total = 6 = 1+2+3

- Total RETURNs: 2 (same for depth 3, 30, 300, or 3,000,000)

English Front-End Proof

The English front-end has compiled all three loop methods above, plus additional methods, to correct Church Machine instruction sequences. Language detection identifies English by the presence of "Add a method", "Set x to", "While", "Repeat with", "Apply lambda with", and other English markers.

PROPOSED CLAIMS

Claim 1 — Self-Invocation via Capability List Register (Independent)

A recursion mechanism in a capability-based processor comprising:

(a) a CALL instruction that, upon entering an abstraction, creates a code region capability register (CR14) and a capability list register (CR6) by splitting the abstraction's memory lump;

(b) wherein the capability list register (CR6) holds an Inform Golden Token pointing to the abstraction's own entry point, providing a self-reference as a free consequence of the CALL lump split — no additional GT allocation, namespace lookup, or c-list modification is required;

(c) wherein a LAMBDA instruction targeting CR6 re-enters the same abstraction with X permission verification only, no namespace gate swap, and no stack frame push — implementing optimal recursive self-invocation with O(1) entry, O(1) context switch, and O(1) exit;

(d) wherein a CALL instruction targeting CR6 also re-enters the same abstraction but with full namespace gate re-validation, a new 2-word stack frame, and O(N) cost — providing no additional security over LAMBDA CR6 because the re-validation re-checks the same GT, re-splits the same lump, and enters the same domain every time;

(e) wherein the architectural analysis demonstrates that LAMBDA CR6 dominates CALL CR6 for self-invocation on every axis: identical security (same method, same domain, same capabilities), but O(1) cost versus O(N) — establishing LAMBDA CR6 as the optimal and preferred recursion primitive.

Claim 2 — Idempotent LAMBDA Re-Entry for O(1) Recursion (Independent)

A hardware mechanism for recursive self-invocation in a capability-based processor, comprising:

(a) a LAMBDA instruction that, when targeting the capability list register (CR6) while the

lambda_active flag is already set, is permitted to re-execute without generating a FAULT — because the return address written to lambda_pc is invariant (always PC+4 of the same LAMBDA CR6 instruction), making the re-entry idempotent;

(b) wherein the method's own recursion argument (e.g., a counter n counting down to zero in data registers) drives the recursion — the hardware does not independently track recursion depth;

(c) wherein the base-case RETURN instruction, upon detecting lambda_active = 1, restores PC from lambda_pc (the instruction after LAMBDA CR6) and clears lambda_active — in a single cycle with no stack access;

(d) wherein a second RETURN instruction, now finding lambda_active = 0, performs the real RETURN by popping the CALL frame from the initial method entry;

(e) wherein exactly two RETURN instructions execute regardless of recursion depth — achieving O(1) exit for arbitrarily deep recursion with no hardware counter, no stack unwinding, and no iterative frame popping;

(f) wherein the FAULT is preserved for LAMBDA to a different target (different CR, different return address) while lambda_active is set — the idempotent exception applies exclusively to CR6 self-invocation;

(g) thereby achieving O(1) recursion entry (one idempotent register write), O(1) context switch (save lambda_active flag + lambda_pc — two fixed-size values regardless of depth), and O(1) exit (two RETURNs always) — using only the existing lambda_active flag and lambda_pc register with no additional hardware.

Claim 3 — Two-RETURN Exit Path (Dependent on Claim 2)

The processor of Claim 2, wherein:

(a) the code structure of a LAMBDA CR6 recursive method places the LAMBDA CR6 instruction as the final executable statement before the method's trailing RETURN instruction;

(b) the base-case RETURN (#1) clears lambda_active and jumps to the instruction after LAMBDA CR6, which is the method's trailing RETURN;

(c) the trailing RETURN (#2) finds lambda_active = 0 and pops the CALL frame from the initial method entry;

(d) thereby, the two-RETURN exit path is a structural consequence of the method layout — the first RETURN navigates to the second RETURN via lambda_pc, and the second RETURN exits the method via the CALL frame — regardless of whether recursion depth was 1, 100, or 1,000,000.

Claim 4 — Three Architectural Loop Styles Demonstrating LAMBDA CR6 Dominance (Independent)

A processor architecture providing three distinct loop mechanisms within a single instruction set, wherein analysis demonstrates that LAMBDA CR6 is the optimal recursion primitive:

(a) a compare-and-branch loop (While) using MCMP and BRANCH instructions, which is familiar to imperative programming but requires branch prediction, is susceptible to pipeline stalls on misprediction, and enables speculative execution vulnerabilities (Spectre, Meltdown) — the exact class of vulnerabilities the Church Machine eliminates;

(b) a recursive repeat (CALL CR6) which invokes the current method through the full CALL path with namespace gate re-validation on every iteration — but wherein the re-validation is redundant because CR6 always points to the same method, re-checking the same GT and re-splitting the same lump, adding O(N) stack and context-switch cost with no security benefit over LAMBDA CR6;

(c) a lambda recursion (LAMBDA CR6) which invokes the current method through the idempotent LAMBDA re-entry path of Claim 2, with X permission check only, no branch prediction, no speculative execution, no namespace swap, no stack frames, and O(1) entry, context switch, and exit — requiring no hardware beyond the existing lambda_active flag and lambda_pc register;

(d) wherein all three mechanisms compile from the same source language to the same instruction set and produce the same computational result, but LAMBDA CR6 dominates: it provides the same security as CALL CR6 (same method, same domain, same capabilities) at O(1) cost instead of O(N), while eliminating all branch-prediction vulnerabilities that afflict While loops;

(e) wherein LAMBDA CR6 requires zero additional hardware — the existing lambda_active flag and lambda_pc register, already present for single-level LAMBDA in the parent application, are the complete recursion mechanism. CALL CR6 recursion exists only as an architectural consequence (CALL can target any CR) and serves as a foil demonstrating the power of LAMBDA CR6.

Claim 5 — Natural Language Compilation to Capability-Secured Instructions (Independent)

A compilation method for a capability-secured processor, comprising:

(a) an English-language front-end that parses structured English sentences including method declarations ("Add a method called X that takes n"), variable assignments ("Set total to total plus n"), conditional blocks ("While n is greater than 0 ... End while", "If n is equal to 0 ... End if"), and recursive self-invocation ("Repeat with n, total", "Apply lambda with n, total");

(b) wherein the English front-end compiles to the same capability-secured instruction set as the JavaScript (imperative) and Haskell (functional) front-ends disclosed in the parent applications;

(c) wherein the English front-end exposes all three loop styles of Claim 4 through natural language syntax: "While ... End while" for compare-and-branch, "Repeat with" for secure recursive repeat, and "Apply lambda with" for lightweight lambda recursion;

(d) wherein the compiled output is indistinguishable from the output of the JavaScript or Haskell front-ends for equivalent computations — the hardware cannot determine the source language;

(e) thereby extending the universal computation target property of the Church Machine instruction set to three programming paradigms: imperative, functional, and natural language.

Claim 6 — Invariant Return Address as Idempotent Re-Entry Precondition (Dependent on Claim 2)

The processor of Claim 2, wherein the correctness of idempotent re-entry depends on a structural invariant:

(a) every LAMBDA CR6 instruction within a given method body branches to the same target (the method's own code entry point, as established by CR6);

(b) every LAMBDA CR6 instruction writes the same return address to lambda_pc (the instruction immediately following the LAMBDA CR6 instruction);

(c) the return address is invariant across all recursion levels — it does not change with recursion depth — making each re-entry write idempotent;

(d) therefore re-entry does not create new return context — the lambda_pc register holds the same value before and after the write — and no stack, counter, or additional state is needed;

(e) this invariant holds exclusively for LAMBDA CR6 self-invocation; LAMBDA to a different target writes a different return address and would corrupt the existing lambda_pc, which is why the non-nestable FAULT is preserved for non-CR6 targets.

Claim 7 — Pet-Name Mathematical Constants via Abstract GT (Dependent on Parent I/O Patent)

A method of representing mathematical constants in a capability-based processor, comprising:

(a) each constant (Pi, E, Phi, Zero, One) is represented as an Abstract Golden Token (gt_type = 11_2) in the abstraction's c-list;

(b) the compiler resolves the constant name to its c-list offset and emits a LOAD instruction to retrieve the Abstract GT into a capability register;

(c) the constant's value is encoded in the Abstract GT's word1_location field and is architecturally immutable — no instruction can modify it;

(d) the constant is unforgeable — no instruction can synthesize a GT, and the constant can only be accessed through capability-mediated LOAD with L permission on CR6;

(e) thereby providing mathematical constants as first-class capability-secured values, accessed through the same GT mechanism used for I/O peripherals and network tunnels.

PRIOR ART DISTINCTION

Recursion in Prior Architectures

System	Self-Invocation	Idempotent Re-Entry	O(1) Context Switch	O(1) Exit	Three Loop Styles	Natural Language	Additional Hardware
x86/ARM	Computed branch	No	No	No	No	No	Branch predictor
MIPS	JAL/JR	No	No	No	No	No	Return address reg
LISP Machines	TCO (compiler)	No	No	No	No	No	Stack
Reduceron	Graph reduction	No	No	No	No	No	Heap
CHERI	Same as base ISA	No	No	No	No	No	Capability cache
CTMM (parent)	CALL only (LAMBDA FAULTs)	No	No	No	No	No	—
CTMM (this CIP)	CALL CR6 + LAMBDA CR6	Yes	Yes	Yes (2 RETURNs)	Yes	Yes	None

Tail-Call Optimization (TCO) — Closest Prior Art

Functional language implementations (Scheme, Haskell, ML) perform tail-call optimization (TCO), which reuses the current stack frame instead of allocating a new one. TCO achieves O(1) stack space for tail-recursive functions.

Distinction: TCO is a compiler optimization that modifies generated code. The present invention is a hardware mechanism that:

1. Does not require compiler participation — the idempotent re-entry operates at the hardware level by relaxing the FAULT condition for CR6
2. Provides O(1) context switch — TCO does not address context switch cost
3. Provides O(1) exit via the two-RETURN path — TCO eliminates the return entirely (tail position), but the present invention preserves the RETURN instruction semantics while making exit cost depth-independent
4. Works within a capability-secured instruction set — TCO operates in unprotected address spaces
5. Requires no additional hardware — the existing lambda_active flag and lambda_pc register, already present for single-level LAMBDA, are the complete mechanism. TCO typically requires compiler analysis and code transformation.

The idempotent re-entry mechanism is architecturally invisible to the instruction stream — the same LAMBDA CR6 instruction executes on each recursion level. The O(1) behavior is a consequence of the return address invariance, not a compiler transformation.

FIGURES (Proposed)

Figure 1: LAMBDA CR6 Idempotent Self-Invocation Flow

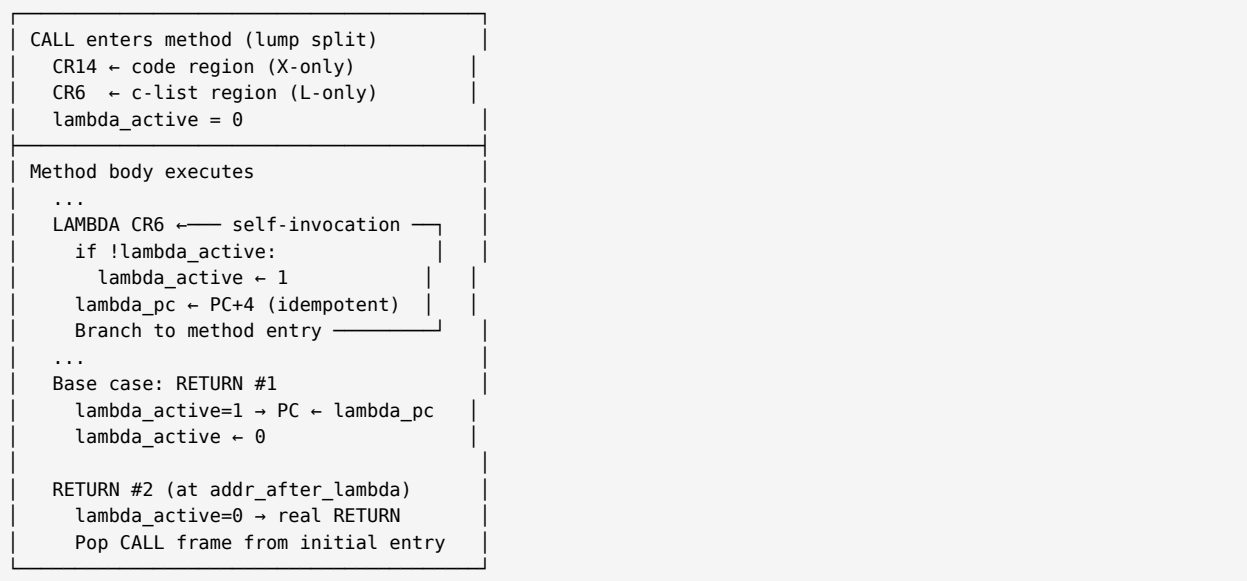


Figure 2: Three Loop Styles — LAMBDA CR6 Dominates

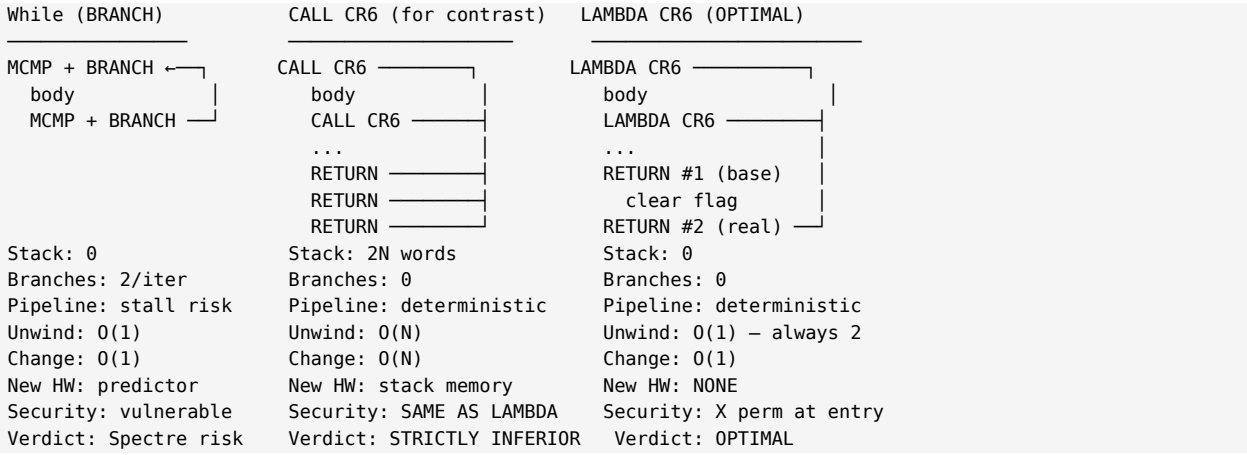
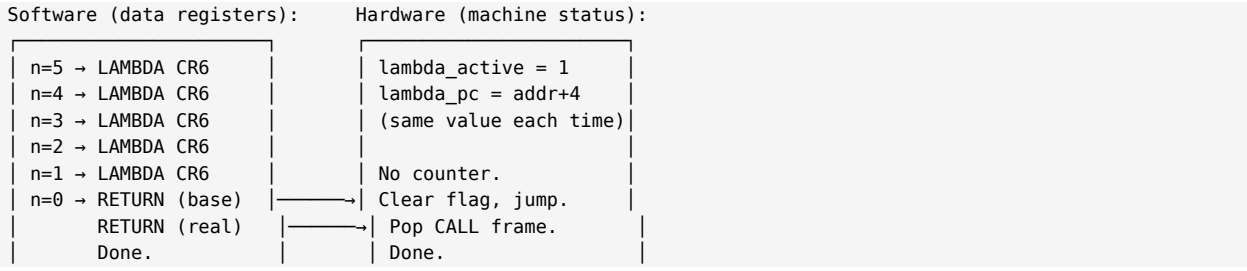


Figure 3: Why No Counter — Software Drives, Hardware Follows



```
└──────────────────────┘       └──────────────────────┘
Software counts.               Hardware doesn't need to.
```

ABSTRACT

A capability-based processor architecture demonstrating that LAMBDA CR6 is the optimal recursion primitive — superior to both conventional branch loops and CALL-based recursion. The CALL instruction's lump split naturally establishes a self-reference in the capability list register (CR6). LAMBDA CR6 exploits this self-reference through idempotent re-entry: LAMBDA CR6 is permitted to re-execute while lambda_active is already set because the return address is invariant — the same value overwrites the same register. The software's own recursion argument drives the countdown; the hardware's existing 1-bit flag and PC register handle exit: the base-case RETURN clears the flag and jumps to the instruction after LAMBDA CR6, where a second RETURN pops the CALL frame. Two RETURNs total, regardless of recursion depth. No hardware counter, no stack frames, no additional registers. CALL CR6 also re-enters the same method but with full namespace gate re-validation — which is redundant because the gate re-checks the same GT and re-splits the same lump every time, adding O(N) cost with no security benefit. LAMBDA CR6 dominates: identical security (same method, same domain, same capabilities), O(1) cost versus O(N), zero additional hardware, and elimination of all branch-prediction vulnerabilities (Spectre, Meltdown, pipeline stalls). The architecture is extended with a natural language (English) front-end for the CLOOMC++ compiler. Pet-name mathematical constants (Pi, E, Phi) are accessed as Abstract Golden Tokens through the capability system.